Farmers Market Database: Answer Key

Companion to: Farmers_Market_SQL_Question_Bank.md · **SQL flavour:** MySQL 8 (default collation `utf8mb4_0900_ai_ci`)

Each question is followed by a model solution and, where useful, the expected result on the supplied data or the reason behind it. Other correct queries are possible. Accept any query that returns the same rows and columns.

Section 1: SELECT & Derived Columns

Q1. From `customer_purchases`, show `product_id`, `customer_id`, `quantity`, `unit_customer_price` and a derived column `total_amount` (quantity × unit price).

Answer

```
SELECT product_id, customer_id, quantity, unit_customer_price,
       quantity * unit_customer_price AS total_amount
FROM customer_purchases;
```

Returns 20 rows. Row 1 has total 40.00 and row 19 has the highest total, 760.00.

Q2. From `customer_purchases`, show each transaction's `total_amount` and `amount_with_tax`, assuming 8% sales tax. Round the tax-inclusive amount to 2 decimal places.

Answer

```
SELECT product_id, customer_id,
       quantity * unit_customer_price                  AS total_amount,
       ROUND(quantity * unit_customer_price * 1.08, 2) AS amount_with_tax
FROM customer_purchases;
```

Example: 40.00 becomes 43.20.

Q3. From `vendor_inventory`, calculate `inventory_value` (quantity × original_price) for every row. Show the rows from highest to lowest inventory value.

Answer

```
SELECT market_date, vendor_id, product_id, quantity, original_price,
       quantity * original_price AS inventory_value
FROM vendor_inventory
ORDER BY inventory_value DESC;
```

The first row is vendor 20 (200 × 34 = 6800.00). ORDER BY can use the alias.

Q4. From `market_date_info`, show each `market_date` with its minimum and maximum temperature, a `temp_range` column (max − min) and an `avg_temp` column (the midpoint of min and max).

Answer

```
SELECT market_date, market_temp_min, market_temp_max,
       market_temp_max - market_temp_min       AS temp_range,
       (market_temp_min + market_temp_max) / 2 AS avg_temp
FROM market_date_info;
```

In MySQL `/` always returns a decimal, so 2024-01-01 gives `avg_temp` = 37.5000.

Q5. Convert `market_temp_min` and `market_temp_max` from Fahrenheit to Celsius using `(F − 32) × 5/9`. Show them as `temp_min_c` and `temp_max_c`, rounded to 1 decimal place.

Answer

```
SELECT market_date,
       ROUND((market_temp_min - 32) * 5 / 9, 1) AS temp_min_c,
       ROUND((market_temp_max - 32) * 5 / 9, 1) AS temp_max_c
FROM market_date_info;
```

Example: 30 °F becomes -1.1 °C.

Q6. From `booth`, show `booth_number`, `booth_price`, `monthly_cost` (booth rented for 4 market days a month) and `annual_cost` (52 market days a year).

Answer

```
SELECT booth_number, booth_price,
       booth_price * 4  AS monthly_cost,
       booth_price * 52 AS annual_cost
FROM booth;
```

Q7. The market plans a 15% markup on all inventory. From `vendor_inventory`, show `original_price`, `new_price` and `price_increase` (the difference between the two). Round both derived columns to 2 decimals.

Answer

```
SELECT market_date, vendor_id, product_id, original_price,
       ROUND(original_price * 1.15, 2)                  AS new_price,
       ROUND(original_price * 1.15 - original_price, 2) AS price_increase
FROM vendor_inventory;
```

Example: 15.00 becomes 17.25, an increase of 2.25.

Q8. A flat $2.50 discount is applied to every unit sold. From `customer_purchases`, show `unit_customer_price`, `discounted_unit_price` and `total_after_discount`.

Answer

```
SELECT product_id, customer_id, quantity, unit_customer_price,
       unit_customer_price - 2.50              AS discounted_unit_price,
       quantity * (unit_customer_price - 2.50) AS total_after_discount
FROM customer_purchases;
```

The brackets are essential. Without them, `quantity * unit_customer_price - 2.50` takes the discount off once instead of per unit.

Q9. From `customer_purchases`, return the columns in this order: customer_id, market_date, transaction_time, product_id and the bill total. Use the aliases **"Customer ID"** and **"Bill Amount"**, which contain spaces.

Answer

```
SELECT customer_id                    AS `Customer ID`,
       market_date, transaction_time, product_id,
       quantity * unit_customer_price  AS `Bill Amount`
FROM customer_purchases;
```

Aliases that contain spaces must be quoted. Backticks are the safest choice in MySQL. `"Customer ID"` also works as an alias, but double quotes mean a *string* in other parts of a MySQL query.

Q10. Can you use an alias defined in the SELECT list inside the WHERE clause of the same query? Write a correct query on `customer_purchases` that returns only transactions whose total (quantity × unit price) is above 400, and show the total as a derived column.

Answer

No. WHERE runs before SELECT, so the alias doesn't exist yet. Repeat the expression instead:

```
SELECT customer_id, product_id,
       quantity * unit_customer_price AS total_amount
FROM customer_purchases
WHERE quantity * unit_customer_price > 400;
```

Returns 8 rows (customers 12–19).

Q11. From `vendor_inventory`, calculate `expected_revenue`, assuming each vendor sells 80% of its stock at the original price. Show only the top 5 rows by expected revenue.

Answer

```
SELECT market_date, vendor_id, product_id,
       quantity * original_price * 0.8 AS expected_revenue
FROM vendor_inventory
ORDER BY expected_revenue DESC
LIMIT 5;
```

Returns vendors 20, 19, 18, 17 and 16, with expected revenue of 5440, 5016, 4608, 4216 and 3840.

Q12. From `booth`, show `booth_price`, `price_per_day` (price covers a 3-day event, rounded to 2 decimals) and `diff_from_base` (booth_price − 100).

Answer

```
SELECT booth_number, booth_price,
       ROUND(booth_price / 3, 2) AS price_per_day,
       booth_price - 100         AS diff_from_base
FROM booth;
```

Q13. From `market_date_info`, show each `market_date` with a column `is_freezing` defined as the expression `market_temp_min < 32` (no CASE). What values does MySQL show in this column, and why?

Answer

```
SELECT market_date, market_temp_min,
       market_temp_min < 32 AS is_freezing
FROM market_date_info;
```

MySQL shows **1** (true) or **0** (false). MySQL has no separate BOOLEAN type: a comparison returns the integer 1, 0, or NULL. 13 of the 20 dates show 1.

Q14. From `market_date_info`, create a derived column `weather_events` = `market_rain_flag + market_snow_flag`. Return only the dates where this value is greater than 0.

Answer

```
SELECT market_date, market_rain_flag, market_snow_flag,
       market_rain_flag + market_snow_flag AS weather_events
FROM market_date_info
WHERE market_rain_flag + market_snow_flag > 0;
```

Returns 5 dates: 01-03, 01-05, 01-10, 01-17 and 01-20. They are all snow days, because the data has no rain days.

Q15. From `customer_purchases`, show the original bill total, the total if each customer had bought **one unit fewer**, and the `revenue_loss` between the two.

Answer

```
SELECT customer_id,
       quantity * unit_customer_price               AS original_total,
       (quantity - 1) * unit_customer_price         AS reduced_total,
       quantity * unit_customer_price
         - (quantity - 1) * unit_customer_price     AS revenue_loss
FROM customer_purchases;
```

The loss always equals `unit_customer_price`. For customer 20 (quantity 1) the reduced total is 0.

Q16. From `vendor`, show `vendor_id`, `vendor_name` and a constant column `market_name` with the value `'Green Valley Farmers Market'` for every row.

Answer

```
SELECT vendor_id, vendor_name,
       'Green Valley Farmers Market' AS market_name
FROM vendor;
```

Q17. From `customer`, show all columns plus two derived columns: `country` (always `'USA'`) and `loyalty_points` (customer_id × 10).

Answer

```
SELECT customer_id, customer_first_name, customer_last_name, customer_zip,
       'USA'            AS country,
       customer_id * 10 AS loyalty_points
FROM customer;
```

You can also write `SELECT *, 'USA' AS country, ...`. MySQL allows `*` when it comes first in the list.

Q18. From `vendor_inventory`, show `quantity`, `full_dozens` (whole dozens available, using `DIV`) and `loose_units` (the units left over, using `%`).

Answer

```
SELECT vendor_id, quantity,
       quantity DIV 12 AS full_dozens,
       quantity % 12   AS loose_units
FROM vendor_inventory;
```

Example: 130 gives 10 full dozens and 10 loose units. `DIV` is integer division; `/` would return 10.8333.

Q19. Compare `quantity + 1 * unit_customer_price` with `(quantity + 1) * unit_customer_price` on `customer_purchases`. Show both as columns and explain why they differ.

Answer

```
SELECT quantity, unit_customer_price,
       quantity + 1 * unit_customer_price   AS without_brackets,
       (quantity + 1) * unit_customer_price AS with_brackets
FROM customer_purchases;
```

`*` has higher precedence than `+`, so the first expression is `quantity + unit_customer_price` (row 1: 2 + 20 = 22). The second is (2 + 1) × 20 = 60.

Q20. In `market_date_info`, `market_year` and `market_week` are VARCHAR columns. Create a derived column `week_code = market_year * 100 + market_week` (e.g. 202401). Does the arithmetic work on string columns? What does MySQL do behind the scenes?

Answer

```
SELECT market_date, market_year, market_week,
       market_year * 100 + market_week AS week_code
FROM market_date_info;
```

Yes, it works: 2024-01-01 gives 202401. MySQL silently converts a string to a number when it is used in arithmetic. The strings are converted to floating-point numbers, and a non-numeric string would become 0 with a warning. This is convenient, but the behaviour is implicit and unsafe to rely on.

Q21. From `booth`, show `booth_price`, `gst_amount` (18% of the price) and `final_price` (price + GST), all rounded to 2 decimals.

Answer

```
SELECT booth_number, booth_price,
       ROUND(booth_price * 0.18, 2) AS gst_amount,
       ROUND(booth_price * 1.18, 2) AS final_price
FROM booth;
```

Q22. Write a query on `vendor` that returns every column but puts `vendor_id` as the **last** column. Why do interviewers prefer explicit column lists to `SELECT *` in production code?

Answer

```
SELECT vendor_name, vendor_type, vendor_owner_first_name,
       vendor_owner_last_name, vendor_id
FROM vendor;
```

Why explicit columns: a schema change (added, removed or reordered columns) can't silently break the code that reads the results; less data is sent over the network; covering indexes can be used; and the query documents itself.

Section 2: Filtering with WHERE

Q23. Find all customers whose zip code is `'12345'`.

Answer

```
SELECT * FROM customer WHERE customer_zip = '12345';
```

Returns Alice Johnson and Kathy Thomas.

Q24. `customer_zip` is VARCHAR. Do `WHERE customer_zip = 12345` and `WHERE customer_zip = '12345'` return the same rows? Now compare `WHERE customer_zip = 01234` with `WHERE customer_zip = '01234'`. Explain what happens in each case.

Answer

- `= 12345` and `= '12345'` return the same two rows. With a number on one side, MySQL converts every `customer_zip` value to a number before comparing. That conversion also stops an index on the column from being used.
- `= 01234` is the number 1234, and `'01234'` converts to 1234, so it matches Jack and Tom. `= '01234'` also matches them, through a plain string comparison.
- The risk: with a numeric comparison, the values `'1234'`, `'01234'` and `' 1234'` would all match. Always compare VARCHAR columns with string literals.

Q25. From `vendor_inventory`, find the rows where `original_price` is at least 25.

Answer

```
SELECT * FROM vendor_inventory WHERE original_price >= 25;
```

Returns 10 rows (vendors 11–20).

Q26. Find all purchases made on `'2024-01-15'`.

Answer

```
SELECT * FROM customer_purchases WHERE market_date = '2024-01-15';
```

Returns 1 row (customer 15).

Q27. Find all purchases whose total (quantity × unit price) exceeds 500, without using a SELECT alias in the WHERE clause.

Answer

```
SELECT *, quantity * unit_customer_price AS total_amount
FROM customer_purchases
WHERE quantity * unit_customer_price > 500;
```

Returns 5 rows: customers 15–19, with totals 544, 595, 648, 703 and 760.

Q28. Find the market dates where the temperature range (max − min) is more than 15 degrees.

Answer

```
SELECT market_date, market_temp_min, market_temp_max,
       market_temp_max - market_temp_min AS temp_range
FROM market_date_info
WHERE market_temp_max - market_temp_min > 15;
```

Returns 6 dates with a range of 16: 01-06, 01-10, 01-14, 01-17, 01-18 and 01-19.

Q29. List the market dates on which it snowed.

Answer

```
SELECT market_date, market_day
FROM market_date_info
WHERE market_snow_flag = 1;
```

Returns 5 dates: 01-03, 01-05, 01-10, 01-17 and 01-20.

Q30. Find the booths whose price is **not** 150. Write the condition twice, once with `!=` and once with `<>`.

Answer

```
SELECT * FROM booth WHERE booth_price != 150;
SELECT * FROM booth WHERE booth_price <> 150;
```

Both return 19 rows (everything except booth 11). `<>` is the ANSI-standard form; `!=` is widely supported.

Q31. Find the purchases made after 6 PM (`transaction_time` later than `'18:00:00'`).

Answer

```
SELECT * FROM customer_purchases WHERE transaction_time > '18:00:00';
```

Returns 5 rows (19:00 to 23:00, customers 10–14). The 18:00 purchase is excluded because `>` is strict. The 00:00–05:00 purchases are "smaller" times, so they don't qualify either.

Q32. Run `WHERE product_size = 'large'` (lowercase) on `product`. Does it return the 'Large' products? What decides whether string comparison is case-sensitive in MySQL?

Answer

Yes, it returns all 'Large' products. MySQL 8's default collation `utf8mb4_0900_ai_ci` is **c**ase-**i**nsensitive (`_ci`). The **collation** of the column or expression decides case sensitivity. To force a case-sensitive match:

```
SELECT * FROM product WHERE BINARY product_size = 'large';            -- 0 rows
SELECT * FROM product WHERE product_size = 'large' COLLATE utf8mb4_bin; -- 0 rows
```

Q33. Data check: find the products whose `product_id` is different from `product_category_id`.

Answer

```
SELECT * FROM product WHERE product_id <> product_category_id;
```

Returns no rows: every product is in the category with the same id. An empty result is a valid answer for a data check.

Q34. Find the market dates where the minimum temperature was below freezing (32 °F).

Answer

```
SELECT market_date, market_day, market_temp_min
FROM market_date_info
WHERE market_temp_min < 32;
```

Returns 13 dates.

Q35. From `vendor_inventory`, find the rows where inventory value (quantity × original_price) exceeds 3000.

Answer

```
SELECT *, quantity * original_price AS inventory_value
FROM vendor_inventory
WHERE quantity * original_price > 3000;
```

Returns 9 rows (vendors 12–20). Vendor 12 is the lowest, at 3120.

Q36. Find the customers with an even `customer_id`, using the `%` operator.

Answer

```
SELECT * FROM customer WHERE customer_id % 2 = 0;
```

Returns 10 customers. `MOD(customer_id, 2) = 0` is equivalent.

Q37. From `customer_purchases`, find the transactions where `unit_customer_price` is more than twice the `quantity`.

Answer

```
SELECT * FROM customer_purchases
WHERE unit_customer_price > 2 * quantity;
```

Returns 17 rows: customers 1–16 and customer 20.

Q38. On `market_date_info`, compare `WHERE market_week = 2` with `WHERE market_week = '2'`. Do both work? Which one is better, and why?

Answer

Both return the 7 week-2 dates.

- `= '2'` compares a string with a string. This is correct, and an index on `market_week` can be used.
- `= 2` makes MySQL convert every `market_week` value to a number. The result is the same, but it prevents index use and can match odd values such as `'2.0'` or `' 2'`.

Prefer `'2'`, because the literal should match the column type.

Q39. Find the purchases where `market_date > '2024-01-10'`. How does MySQL compare a DATE column with a string literal?

Answer

```
SELECT * FROM customer_purchases WHERE market_date > '2024-01-10';
```

Returns 10 rows (01-11 to 01-20). MySQL converts the string literal to a DATE, so the comparison is chronological, not alphabetical. Use the unambiguous `'YYYY-MM-DD'` format.

Q40. What is wrong with `SELECT * FROM customer WHERE customer_zip = NULL;`? Rewrite it correctly and explain why `= NULL` never matches.

Answer

`= NULL` compares with an unknown value, so the result is always NULL (unknown), never TRUE. It returns no rows even where zip is NULL.

```
SELECT * FROM customer WHERE customer_zip IS NULL;
```

This also returns 0 rows here, because no zip is missing. `IS NULL` is still the correct test.

Section 3: AND / OR / NOT

Q41. Find the vendors of `'Type B'` whose owner's last name is `'White'`.

Answer

```
SELECT * FROM vendor
WHERE vendor_type = 'Type B' AND vendor_owner_last_name = 'White';
```

Returns vendors 4 (Bob White) and 16 (Nick White).

Q42. Find the Winter market dates that had snow **and** a minimum temperature below 30.

Answer

```
SELECT * FROM market_date_info
WHERE market_season = 'Winter'
  AND market_snow_flag = 1
  AND market_temp_min < 30;
```

Returns 4 dates: 01-03, 01-05, 01-10 and 01-17. 01-20 is excluded because its minimum is 35.

Q43. Find the market dates that had rain **or** snow.

Answer

```
SELECT * FROM market_date_info
WHERE market_rain_flag = 1 OR market_snow_flag = 1;
```

Returns 5 dates, all snow days.

Q44. Find the purchases where quantity is more than 10 **and** unit price is below 35.

Answer

```
SELECT * FROM customer_purchases
WHERE quantity > 10 AND unit_customer_price < 35;
```

Returns 6 rows (customers 10–15).

Q45. Find the booths that are either a `'Large booth'` or priced above 180.

Answer

```
SELECT * FROM booth
WHERE booth_description = 'Large booth' OR booth_price > 180;
```

Returns 9 booths: 1, 4, 7, 10, 13, 16, 18, 19 and 20.

Q46. What will this query return?

```
SELECT * FROM vendor
WHERE vendor_type = 'Type A' OR vendor_type = 'Type B' AND vendor_id > 15;
```

Rewrite it with parentheses so that it returns only vendors with `vendor_id > 15`, of either type.

Answer

AND is evaluated before OR, so the query means `Type A OR (Type B AND id > 15)`. It returns **13 rows**: all 10 Type A vendors plus 16, 18 and 20.

```
SELECT * FROM vendor
WHERE (vendor_type = 'Type A' OR vendor_type = 'Type B')
  AND vendor_id > 15;
```

The corrected query returns vendors 16–20.

Q47. Find the customers whose zip is `'12345'` or `'23456'` and whose `customer_id` is greater than 10. Watch the operator precedence.

Answer

```
SELECT * FROM customer
WHERE (customer_zip = '12345' OR customer_zip = '23456')
  AND customer_id > 10;
```

Returns Kathy (11) and Leo (12). Without the brackets you would also get Alice (1).

Q48. Using `NOT` and `OR` (no IN), list the market dates that are **not** Saturday or Sunday.

Answer

```
SELECT * FROM market_date_info
WHERE NOT (market_day = 'Saturday' OR market_day = 'Sunday');
```

Returns 15 dates.

Q49. Rewrite `WHERE NOT (quantity < 50 OR original_price > 30)` on `vendor_inventory` **without** using NOT, applying De Morgan's law. Check that both versions return the same rows.

Answer

By De Morgan's law, NOT (A OR B) = (NOT A) AND (NOT B):

```
SELECT * FROM vendor_inventory
WHERE quantity >= 50 AND original_price <= 30;
```

Both versions return 12 rows (vendors 5–16).

Q50. Using only comparison operators and AND (no BETWEEN), find the purchases made from 10:00 AM to 4:00 PM (inclusive) with quantity of at least 5.

Answer

```
SELECT * FROM customer_purchases
WHERE transaction_time >= '10:00:00'
  AND transaction_time <= '16:00:00'
  AND quantity >= 5;
```

Returns 4 rows (customers 4–7).

Q51. Find the products that are 'Small' or 'Medium' **and** belong to a category id greater than 5.

Answer

```
SELECT * FROM product
WHERE product_size IN ('Small', 'Medium')      -- or (size='Small' OR size='Medium')
  AND product_category_id > 5;
```

Returns 9 products: 7, 8, 10, 11, 13, 14, 16, 17 and 19.

Q52. Find the market dates where the minimum temperature was below 30 or the maximum was above 48, **but** there was no snow.

Answer

```
SELECT * FROM market_date_info
WHERE (market_temp_min < 30 OR market_temp_max > 48)
  AND market_snow_flag = 0;
```

Returns 4 dates: 01-09, 01-12, 01-15 and 01-18.

Q53. Find the customers with `customer_id` greater than 5 and less than 15 whose zip is not `'45678'`.

Answer

```
SELECT * FROM customer
WHERE customer_id > 5 AND customer_id < 15
  AND customer_zip <> '45678';
```

Returns 8 customers (ids 6–13). Customer 14, Nick, is excluded.

Q54. Find the booths that are either Medium booths priced over 150, or Small booths priced under 130.

Answer

```
SELECT * FROM booth
WHERE (booth_description = 'Medium booth' AND booth_price > 150)
   OR (booth_description = 'Small booth'  AND booth_price < 130);
```

Returns booths 14, 17 and 20 (Medium) and 3 and 6 (Small).

Q55. Find the purchases where (quantity ≥ 15 and unit price ≥ 35) **or** quantity ≤ 3.

Answer

```
SELECT * FROM customer_purchases
WHERE (quantity >= 15 AND unit_customer_price >= 35)
   OR quantity <= 3;
```

Returns 7 rows: customers 16–19 and customers 1, 2 and 20.

Q56. Compare these two filters on `market_date_info`:

- `WHERE NOT market_rain_flag = 1 AND NOT market_snow_flag = 1`
- `WHERE NOT (market_rain_flag = 1 AND market_snow_flag = 1)`

Do they return the same dates? Explain the difference.

Answer

They are **different**:

- The first is `(NOT rain) AND (NOT snow)`, which means "neither rain nor snow". It returns the **15** non-snow dates.
- The second is `NOT (rain AND snow)`, which means "not both at once". By De Morgan this is `(NOT rain) OR (NOT snow)`, and it returns all **20** dates, because no date has both flags.

Q57. Find the vendors where **exactly one** of these is true: the owner's last name is `'White'`, or the vendor type is `'Type A'`. Rows where both are true should not appear.

Answer

```
SELECT * FROM vendor
WHERE (vendor_owner_last_name = 'White' OR vendor_type = 'Type A')
  AND NOT (vendor_owner_last_name = 'White' AND vendor_type = 'Type A');
-- MySQL shortcut: WHERE (vendor_owner_last_name = 'White') XOR (vendor_type = 'Type A')
```

Returns 12 vendors: all 10 Type A vendors plus 4 and 16. No White is Type A, so no row is removed for matching both.

Q58. Find the Tuesdays or Thursdays in market week 2 where the maximum temperature was at least 45.

Answer

```
SELECT * FROM market_date_info
WHERE (market_day = 'Tuesday' OR market_day = 'Thursday')
  AND market_week = '2'
  AND market_temp_max >= 45;
```

Returns 01-09 (Tuesday, 50) and 01-11 (Thursday, 45).

Q59. From `vendor_inventory`, find the rows for vendor ids 5 to 10 (inclusive) where the original price is not 20. Use only AND/NOT and comparison operators.

Answer

```
SELECT * FROM vendor_inventory
WHERE vendor_id >= 5 AND vendor_id <= 10
  AND NOT original_price = 20;
```

Returns vendors 5, 7, 8, 9 and 10.

Q60. Find the purchases that are either morning purchases (before 12:00) with a total above 100, or evening purchases (after 18:00) with a total above 300.

Answer

```
SELECT *, quantity * unit_customer_price AS total_amount
FROM customer_purchases
WHERE (transaction_time < '12:00:00' AND quantity * unit_customer_price > 100)
   OR (transaction_time > '18:00:00' AND quantity * unit_customer_price > 300);
```

Returns 10 rows: customers 15–19 (00:00–04:00 is "before 12") and customers 10–14.

Q61. What will this return?

```
SELECT NULL AND 0, NULL AND 1, NULL OR 1, NULL OR 0, NOT NULL;
```

Answer

Expression	Result	Why
`NULL AND 0`	**0**	FALSE AND anything is FALSE
`NULL AND 1`	**NULL**	Unknown
`NULL OR 1`	**1**	TRUE OR anything is TRUE
`NULL OR 0`	**NULL**	Unknown
`NOT NULL`	**NULL**	NOT unknown is still unknown

Q62. Find the booths that are **not** Large booths **and** have an odd booth number.

Answer

```
SELECT * FROM booth
WHERE NOT booth_description = 'Large booth'
  AND booth_number % 2 = 1;
```

Returns booths 3, 5, 9, 11, 15 and 17.

Section 4: IN / NOT IN

Q63. Find the customers whose zip code is `'12345'`, `'56789'` or `'90123'`.

Answer

```
SELECT * FROM customer
WHERE customer_zip IN ('12345', '56789', '90123');
```

Returns 6 customers: Alice, Kathy, Eve, Olive, Ivy and Steve.

Q64. Using IN, list the market dates that fall on a Saturday or Sunday.

Answer

```
SELECT * FROM market_date_info
WHERE market_day IN ('Saturday', 'Sunday');
```

Returns 5 dates: 01-06, 01-07, 01-13, 01-14 and 01-20.

Q65. Find the vendors whose owner's last name is Smith, Brown, White or Jones.

Answer

```
SELECT * FROM vendor
WHERE vendor_owner_last_name IN ('Smith', 'Brown', 'White', 'Jones');
```

Returns vendors 2, 3, 4, 7 and 16.

Q66. Find the products whose size is **not** 'Small' or 'Large'.

Answer

```
SELECT * FROM product WHERE product_size NOT IN ('Small', 'Large');
```

Returns 6 Medium products: 2, 5, 8, 11, 14 and 17. Product 20 is 'Large'.

Q67. Find all purchases made on `'2024-01-03'`, `'2024-01-09'`, `'2024-01-14'` or `'2024-01-20'`.

Answer

```
SELECT * FROM customer_purchases
WHERE market_date IN ('2024-01-03', '2024-01-09', '2024-01-14', '2024-01-20');
```

Returns 4 rows.

Q68. From `vendor_inventory`, find the rows where `product_id` is not 1, 5, 10, 15 or 20.

Answer

```
SELECT * FROM vendor_inventory WHERE product_id NOT IN (1, 5, 10, 15, 20);
```

Returns 15 rows.

Q69. Find the booths whose size is `'10x10'` or `'6x6'`, excluding booth numbers 1, 3 and 19.

Answer

```
SELECT * FROM booth
WHERE booth_size IN ('10x10', '6x6')
  AND booth_number NOT IN (1, 3, 19);
```

Returns 10 booths: 4, 6, 7, 9, 10, 12, 13, 15, 16 and 18.

Q70. What will this return, and why?

```
SELECT * FROM customer WHERE customer_id NOT IN (1, 2, NULL);
```

Answer

Returns **no rows**. `x NOT IN (1, 2, NULL)` expands to `x <> 1 AND x <> 2 AND x <> NULL`. The last comparison is always NULL, so the whole condition can never be TRUE. This is the classic trap with `NOT IN`

on a list or subquery that contains NULL.

Q71. What will this return? Compare it with Q70 and explain the difference.

```
SELECT * FROM customer WHERE customer_id IN (1, 2, NULL);
```

Answer

Returns customers **1 and 2**. IN is an OR chain: `x = 1 OR x = 2 OR x = NULL`. One TRUE branch is enough, so the NULL is harmless here. NULL only causes damage with NOT IN.

Q72. Find the customers whose zip code **starts** with 1, 2 or 3, using LEFT together with IN.

Answer

```
SELECT * FROM customer
WHERE LEFT(customer_zip, 1) IN ('1', '2', '3');
```

Returns 6 customers: Alice, Bob, Charlie, Kathy, Leo and Mona.

Q73. Find the weekday market dates (Monday to Friday, using IN) on which it snowed.

Answer

```
SELECT * FROM market_date_info
WHERE market_day IN ('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday')
  AND market_snow_flag = 1;
```

Returns 01-03, 01-05, 01-10 and 01-17. 01-20 is a Saturday.

Q74. Find the vendors whose owner's first name is Alice, Bob, Eve or Grace. Show the owner's full name as one column.

Answer

```
SELECT vendor_id,
       CONCAT(vendor_owner_first_name, ' ', vendor_owner_last_name) AS owner_name
FROM vendor
WHERE vendor_owner_first_name IN ('Alice', 'Bob', 'Eve', 'Grace');
```

Returns Alice Brown, Bob White, Eve Jones and Grace Miller.

Q75. Rewrite this with IN, then say whether the two versions differ in result or readability:

```
WHERE market_day = 'Monday' OR market_day = 'Wednesday' OR market_day = 'Friday'
```

Answer

```
WHERE market_day IN ('Monday', 'Wednesday', 'Friday')
```

The result is identical: IN is shorthand for an OR chain of `=` tests. IN is shorter, easier to maintain and less likely to cause precedence bugs when combined with AND. MySQL sorts constant IN lists and uses a binary search, so long lists are efficient.

Q76. Find the purchases whose `transaction_time` is `'10:00:00'`, `'12:00:00'` or `'14:00:00'`. Would the query still work if the values were written as `'10:00'`, `'12:00'`, `'14:00'`?

Answer

```
SELECT * FROM customer_purchases
WHERE transaction_time IN ('10:00:00', '12:00:00', '14:00:00');
```

Returns customers 1, 3 and 5. **Yes**, `'10:00'` also works. The constants are converted to TIME for the comparison, and `'10:00'` is read as 10:00:00 (HH:MM).

Q77. Find the products in categories 1, 3, 5, 7 or 9 whose size is Medium or Large.

Answer

```
SELECT * FROM product
WHERE product_category_id IN (1, 3, 5, 7, 9)
  AND product_size IN ('Medium', 'Large');
```

Returns products 3, 5 and 9.

Q78. Find the customers whose zip code **ends** in 0 or 5, using RIGHT with IN.

Answer

```
SELECT * FROM customer WHERE RIGHT(customer_zip, 1) IN ('0', '5');
```

Returns Alice and Kathy (12345) and Frank and Paul (67890).

Q79. `market_week` is VARCHAR. Does `WHERE market_week IN (1, 3)` work? What type conversion happens?

Answer

Yes, it works. The list values are numbers, so MySQL compares numerically and converts each `market_week` string to a number. It returns the 13 dates in weeks 1 and 3. As in Q38, `IN ('1', '3')` is the cleaner, index-friendly version.

Q80. Find the booths whose description is 'Large booth' or 'Small booth' and whose price is neither 100 nor 190.

Answer

```
SELECT * FROM booth
WHERE booth_description IN ('Large booth', 'Small booth')
  AND booth_price NOT IN (100, 190);
```

Returns 11 booths. Booths 1 and 19 are excluded.

Section 5: BETWEEN

Q81. Find all purchases between `'2024-01-05'` and `'2024-01-10'`. Are both end dates included? How many rows do you expect?

Answer

```
SELECT * FROM customer_purchases
WHERE market_date BETWEEN '2024-01-05' AND '2024-01-10';
```

BETWEEN includes **both** ends, so it returns **6 rows** (01-05 to 01-10).

Q82. Find the booths priced between 120 and 160.

Answer

```
SELECT * FROM booth WHERE booth_price BETWEEN 120 AND 160;
```

Returns 9 booths (5–13).

Q83. From `vendor_inventory`, find the rows where quantity is **not** between 50 and 150.

Answer

```
SELECT * FROM vendor_inventory WHERE quantity NOT BETWEEN 50 AND 150;
```

Returns 9 rows: quantities 10–40 and 160–200.

Q84. What will `SELECT * FROM booth WHERE booth_price BETWEEN 160 AND 120;` return? Explain.

Answer

No rows. `BETWEEN a AND b` means `>= a AND <= b`. With a = 160 and b = 120, no value can be at least 160 and at most 120. The lower bound must come first.

Q85. Find the purchases made between 9 AM and 5 PM using BETWEEN on `transaction_time`.

Answer

```
SELECT * FROM customer_purchases
WHERE transaction_time BETWEEN '09:00:00' AND '17:00:00';
```

Returns 8 rows (10:00 to 17:00, customers 1–8).

Q86. The market wants the "late-night" transactions from 10 PM to 4 AM. Why does `transaction_time BETWEEN '22:00:00' AND '04:00:00'` fail? Write a query that works.

Answer

Because the lower bound (22:00) is greater than the upper bound (04:00), `BETWEEN '22:00:00' AND '04:00:00'` is empty, the same problem as in Q84. A window that crosses midnight must be split with OR:

```
SELECT * FROM customer_purchases
WHERE transaction_time >= '22:00:00'
   OR transaction_time <= '04:00:00';
```

Returns 7 rows (22:00, 23:00, 00:00, 01:00, 02:00, 03:00 and 04:00).

Q87. Does `WHERE customer_last_name BETWEEN 'A' AND 'H'` include 'Harris'? Explain why, and fix the query so that all last names starting with A to H are included.

Answer

No. Strings are compared character by character, and `'Harris'` > `'H'` because a longer string with the same prefix sorts after the shorter one. The query returns only Anderson, Brown, Clark and Davis. Fix:

```
SELECT * FROM customer
WHERE customer_last_name >= 'A' AND customer_last_name < 'I';
```

This returns 5 rows, including Harris.

Q88. What will this return, and why?

```
SELECT DISTINCT market_week FROM market_date_info
WHERE market_week BETWEEN '1' AND '10';
```

Answer

It returns only **'1'**. Both sides are strings, so the comparison is alphabetical: '1' ≤ '1' ≤ '10' is true, but '2' and '3' are greater than '10' as strings, because the first characters '2' and '3' come after '1'. Use numbers (`BETWEEN 1 AND 10`) or store week numbers in a numeric column.

Q89. Find the purchases whose total (quantity × unit price) is between 200 and 500.

Answer

```
SELECT *, quantity * unit_customer_price AS total_amount
FROM customer_purchases
WHERE quantity * unit_customer_price BETWEEN 200 AND 500;
```

Returns 8 rows: customers 7–14, with totals from 208 to 495.

Q90. Find the market dates where the temperature range (max − min) is between 14 and 15.

Answer

```
SELECT market_date, market_temp_max - market_temp_min AS temp_range
FROM market_date_info
WHERE market_temp_max - market_temp_min BETWEEN 14 AND 15;
```

Returns 13 dates.

Q91. From `vendor_inventory`, find the rows with original price between 20 and 30 **and** market date between `'2024-01-01'` and `'2024-01-12'`.

Answer

```
SELECT * FROM vendor_inventory
WHERE original_price BETWEEN 20 AND 30
  AND market_date BETWEEN '2024-01-01' AND '2024-01-12';
```

Returns 7 rows (vendors 6–12).

Q92. Suppose `market_date` were a DATETIME column holding values like `'2024-01-10 15:30:00'`. Which rows would `BETWEEN '2024-01-05' AND '2024-01-10'` miss? Write a safer version of the filter.

Answer

`BETWEEN '2024-01-05' AND '2024-01-10'` becomes `<= '2024-01-10 00:00:00'`, so every row from **after midnight on 10 January** is lost. A safer, half-open version:

```
WHERE market_date >= '2024-01-05'
  AND market_date <  '2024-01-11'
```

Q93. Find the customers with `customer_id` between 5 and 15, excluding customer 10.

Answer

```
SELECT * FROM customer
WHERE customer_id BETWEEN 5 AND 15
  AND customer_id <> 10;
```

Returns 10 customers.

Q94. What will this return, and in which order would the names sort?

```
SELECT product_name FROM product
WHERE product_name BETWEEN 'Product 1' AND 'Product 2';
```

Answer

It returns **12 rows**, in this order: 'Product 1', 'Product 10', 'Product 11', ..., 'Product 19', 'Product 2'. As strings, 'Product 10' sorts before 'Product 2', and 'Product 20' is greater than 'Product 2', so it is excluded.

This is the classic trap of sorting numbers stored as text.

Q95. Find the booths numbered 5 to 15 whose price is **not** between 130 and 150.

Answer

```
SELECT * FROM booth
WHERE booth_number BETWEEN 5 AND 15
  AND booth_price NOT BETWEEN 130 AND 150;
```

Returns booths 5, 6, 12, 13, 14 and 15.

Q96. Find the market dates where the average of min and max temperature is between 38 and 40.

Answer

```
SELECT market_date, (market_temp_min + market_temp_max) / 2 AS avg_temp
FROM market_date_info
WHERE (market_temp_min + market_temp_max) / 2 BETWEEN 38 AND 40;
```

Returns 6 dates: 01-02, 01-06, 01-07, 01-13, 01-14 and 01-19.

Q97. Using string comparison, find the customers whose zip is between `'20000'` and `'60000'`. Where would `'01234'` fall, and why?

Answer

```
SELECT * FROM customer
WHERE customer_zip BETWEEN '20000' AND '60000';
```

Returns 8 customers (zips 23456, 34567, 45678 and 56789, two customers each). `'01234'` starts with '0', which sorts before '2', so it falls **below** the range. It is compared as text, not as the number 1234.

Q98. Using RIGHT on `market_date` together with BETWEEN, find the market dates that fall in the first half of the month (day 1 to 15).

Answer

```
SELECT market_date, market_day
FROM market_date_info
WHERE RIGHT(market_date, 2) BETWEEN 1 AND 15;
```

Returns 15 dates. `RIGHT(DATE, 2)` treats the date as the text 'YYYY-MM-DD' and returns '01' to '20'. Comparing that with numbers converts it to 1–20. `BETWEEN '01' AND '15'` also works here, because both strings are two characters long.

Section 6: Pattern Matching with LIKE

Q99. Find the customers whose first name starts with a vowel.

Answer

```
SELECT * FROM customer
WHERE customer_first_name LIKE 'A%' OR customer_first_name LIKE 'E%'
   OR customer_first_name LIKE 'I%' OR customer_first_name LIKE 'O%'
   OR customer_first_name LIKE 'U%';
```

Returns Alice, Eve, Ivy and Olive. `LEFT(customer_first_name, 1) IN ('A','E','I','O','U')` is a shorter alternative.

Q100. Find the customers whose last name ends with `'son'`.

Answer

```
SELECT * FROM customer WHERE customer_last_name LIKE '%son';
```

Returns Johnson, Wilson, Anderson, Jackson and Robinson.

Q101. Using the `_` wildcard, find the customers whose first name has exactly 3 characters.

Answer

```
SELECT * FROM customer WHERE customer_first_name LIKE '___';
```

Returns Bob, Eve, Ivy, Leo and Tom. Each `_` matches exactly one character.

Q102. Find the customers whose first name has `'a'` as its second letter.

Answer

```
SELECT * FROM customer WHERE customer_first_name LIKE '_a%';
```

Returns David, Hank, Jack, Kathy and Paul.

Q103. Find the customers whose last name contains `'o'` but does **not** start with `'O'`.

Answer

```
SELECT * FROM customer
WHERE customer_last_name LIKE '%o%'
  AND customer_last_name NOT LIKE 'O%';
```

Returns 10 customers: Johnson, Brown, Wilson, Jones, Moore, Taylor, Anderson, Thomas, Jackson and Robinson.

Q104. Work out how many rows each condition returns on `product`, then check:

- `product_name LIKE '%1%'`
- `product_name LIKE '%1'`
- `product_name LIKE 'Product 1_'`

Answer

Condition	Rows	Which
`LIKE '%1%'`	**11**	1, 10–19
`LIKE '%1'`	**2**	1, 11
`LIKE 'Product 1_'`	**10**	10–19 (exactly one character after "1")

Q105. Using LIKE only, find the products whose name ends with a two-digit number.

Answer

```
SELECT * FROM product WHERE product_name LIKE 'Product __';
```

Returns 11 products (10–20). `_` matches any character, so `'Product ab'` would also match. LIKE can't restrict a position to digits (REGEXP can).

Q106. Find the product SKUs that are exactly 4 characters long, using LIKE only.

Answer

```
SELECT * FROM product WHERE product_sku LIKE '____';
```

Returns SKU1 to SKU9 (9 rows).

Q107. Find the market days whose name contains `'es'`.

Answer

```
SELECT * FROM market_date_info WHERE market_day LIKE '%es%';
```

Returns the Tuesdays and Wednesdays (6 dates).

Q108. Find the booths whose size starts with `'1'`.

Answer

```
SELECT * FROM booth WHERE booth_size LIKE '1%';
```

Returns all the '10x10' booths (7 rows).

Q109. Find the customers whose zip code starts with `'0'`. Why is the leading zero kept?

Answer

```
SELECT * FROM customer WHERE customer_zip LIKE '0%';
```

Returns Jack and Tom. The column is VARCHAR, so '01234' is stored as text. In an INT column the leading zero would be lost (1234). That is why zip codes, phone numbers and IDs with leading zeros belong in string columns.

Q110. What does `WHERE vendor_name LIKE '%_%'` return, and why? If some vendor names contained a real underscore (e.g. `'Vendor_21'`), how would you search for the literal `_`?

Answer

`_` is a wildcard for "exactly one character", so `'%_%'` means "at least one character" and returns **all 20 vendors**. To search for a literal underscore, escape it:

```
SELECT * FROM vendor WHERE vendor_name LIKE '%\_%';
-- or with an explicit escape character:
SELECT * FROM vendor WHERE vendor_name LIKE '%!_%' ESCAPE '!';
```

Q111. Find the vendors whose owner's first name does **not** start with a vowel.

Answer

```
SELECT * FROM vendor
WHERE vendor_owner_first_name NOT LIKE 'A%'
  AND vendor_owner_first_name NOT LIKE 'E%'
  AND vendor_owner_first_name NOT LIKE 'I%'
  AND vendor_owner_first_name NOT LIKE 'O%'
  AND vendor_owner_first_name NOT LIKE 'U%';
```

Returns 16 vendors. The excluded owners are Alice, Eve, Ivy and Olive. Note the **AND**: with NOT LIKE, joining the conditions with OR would return every row.

Q112. Find the customers whose last name contains a double letter (`ll`, `rr`, `oo`, `ee`, `ss`, `nn` or `tt`).

Answer

```
SELECT * FROM customer
WHERE customer_last_name LIKE '%ll%' OR customer_last_name LIKE '%rr%'
   OR customer_last_name LIKE '%oo%' OR customer_last_name LIKE '%ee%'
   OR customer_last_name LIKE '%ss%' OR customer_last_name LIKE '%nn%'
   OR customer_last_name LIKE '%tt%';
```

Returns Miller, Moore, Harris and Lee.

Q113. Does `WHERE customer_first_name LIKE 'a%'` return 'Alice'? How would you make the match case-sensitive?

Answer

Yes, because the default collation is case-insensitive. To make the match case-sensitive:

```
SELECT * FROM customer WHERE customer_first_name LIKE BINARY 'a%';               -- 0 rows
SELECT * FROM customer WHERE customer_first_name LIKE 'a%' COLLATE utf8mb4_bin;  -- 0 rows
```

Q114. Find the customers whose first name ends with `'a'`.

Answer

```
SELECT * FROM customer WHERE customer_first_name LIKE '%a';
```

Returns Mona and Rita.

Q115. Find the customers whose first name starts with a consonant **and** ends with a vowel.

Answer

```
SELECT * FROM customer
WHERE customer_first_name NOT LIKE 'A%' AND customer_first_name NOT LIKE 'E%'
  AND customer_first_name NOT LIKE 'I%' AND customer_first_name NOT LIKE 'O%'
  AND customer_first_name NOT LIKE 'U%'
  AND (customer_first_name LIKE '%a' OR customer_first_name LIKE '%e'
    OR customer_first_name LIKE '%i' OR customer_first_name LIKE '%o'
    OR customer_first_name LIKE '%u');
```

Returns Charlie, Grace, Leo, Mona, Rita and Steve.

Q116. Using LIKE on `transaction_time`, find the purchases made from 10:00 to 19:59. What does MySQL do when LIKE is used on a TIME column?

Answer

```
SELECT * FROM customer_purchases WHERE transaction_time LIKE '1_:%';
```

Returns 10 rows (10:00 to 19:00). LIKE works on strings, so MySQL first converts the TIME value to the text 'HH:MM:SS' and then matches the pattern. It works, but an index can't be used. `transaction_time BETWEEN '10:00:00' AND '19:59:59'` is the better version.

Q117. Using LIKE on `market_date`, find the market dates from the 10th to the 19th of the month.

Answer

```
SELECT * FROM market_date_info WHERE market_date LIKE '2024-01-1_';
```

Returns 10 dates (01-10 to 01-19).

Q118. Find the vendors whose owner's full name (first + last, joined inside the query) contains `'an'`.

Answer

```
SELECT vendor_id,
       CONCAT(vendor_owner_first_name, ' ', vendor_owner_last_name) AS owner_name
FROM vendor
WHERE CONCAT(vendor_owner_first_name, ' ', vendor_owner_last_name) LIKE '%an%';
```

Returns Jane Smith, Frank Davis, Hank Wilson and Kathy Anderson. 'An' in Anderson matches because the comparison is case-insensitive.

Q119. Find the customers whose last name has `'r'` as the **third-last** letter.

Answer

```
SELECT * FROM customer WHERE customer_last_name LIKE '%r__';
```

Returns Harris and Perez.

Q120. Compare `vendor_name = 'Vendor 1 '` with `vendor_name LIKE 'Vendor 1 '` (note the trailing space). Do they return the same rows? Explain how trailing spaces are handled by `=` and by `LIKE`.

Answer

It depends on the collation's **pad attribute**:

- MySQL 8's default `utf8mb4_0900_ai_ci` is **NO PAD**. Trailing spaces count, so `= 'Vendor 1 '` and `LIKE 'Vendor 1 '` both return **no rows**.
- Older **PAD SPACE** collations (`utf8mb4_general_ci`, `latin1_swedish_ci`, MySQL 5.7 defaults) ignore trailing spaces with `=`, so `=` returns *Vendor 1*. `LIKE` still returns nothing, because LIKE compares character by character and never pads.

You can check the attribute with `SELECT COLLATION_NAME, PAD_ATTRIBUTE FROM information_schema.COLLATIONS`.

Q121. Find the products whose name contains neither `'1'` nor `'2'`.

Answer

```
SELECT * FROM product
WHERE product_name NOT LIKE '%1%' AND product_name NOT LIKE '%2%';
```

Returns Products 3–9 (7 rows).

Q122. Find the customers whose first name has exactly 5 letters and contains `'i'`.

Answer

```
SELECT * FROM customer
WHERE customer_first_name LIKE '_____'
  AND customer_first_name LIKE '%i%';
```

Returns Alice, David and Olive.

Section 7: CONCAT

Q123. Show each customer's name as **"Last, First"** (e.g. `Johnson, Alice`).

Answer

```
SELECT CONCAT(customer_last_name, ', ', customer_first_name) AS customer_name
FROM customer;
```

Example output: `Johnson, Alice`.

Q124. Show vendor details as **"John Doe (Vendor 1)"**.

Answer

```
SELECT CONCAT(vendor_owner_first_name, ' ', vendor_owner_last_name,
              ' (', vendor_name, ')') AS vendor_details
FROM vendor;
```

Q125. Create a customer label in the format **"Alice Johnson - 12345"**.

Answer

```
SELECT CONCAT(customer_first_name, ' ', customer_last_name, ' - ', customer_zip) AS
customer_label
FROM customer;
```

Q126. Generate an email ID for every customer in the format `firstname.lastname@farmersmarket.com`, all in lowercase.

Answer

```
SELECT LOWER(CONCAT(customer_first_name, '.', customer_last_name, '@farmersmarket.com')) AS
email
FROM customer;
```

Example output: `alice.johnson@farmersmarket.com`.

Q127. Create a booth label in the format **"#1 | Large booth | 10x10 | $100.00"**.

Answer

```
SELECT CONCAT('#', booth_number, ' | ', booth_description, ' | ',
              booth_size, ' | $', booth_price) AS booth_label
FROM booth;
```

`booth_price` is DECIMAL(10,2), so it prints as `100.00`.

Q128. Create a vendor code in the format **"A-001"**: the type letter from `vendor_type`, a hyphen, then `vendor_id` zero-padded to 3 digits. Don't use LPAD. Hint: CONCAT and RIGHT.

Answer

```
SELECT vendor_id,
       CONCAT(RIGHT(vendor_type, 1), '-', RIGHT(CONCAT('000', vendor_id), 3)) AS vendor_code
FROM vendor;
```

Returns `A-001`, `B-002`, ..., `B-020`. Adding zeros in front and keeping the right-most 3 characters is the classic padding trick when LPAD isn't allowed.

Q129. From `market_date_info`, create the sentence **"Monday, 2024-01-01: 30°F to 45°F"** for every date.

Answer

```
SELECT CONCAT(market_day, ', ', market_date, ': ',
              market_temp_min, '°F to ', market_temp_max, '°F') AS weather_line
FROM market_date_info;
```

Q130. What will `SELECT CONCAT('Vendor', NULL, 'X');` return? How would you protect a CONCAT against NULL values?

Answer

It returns **NULL**: if any argument of CONCAT is NULL, the whole result is NULL. To protect against it, wrap nullable columns as `CONCAT('Vendor', IFNULL(col, ''), 'X')` (or COALESCE), or use `CONCAT_WS`, which skips NULL values.

Q131. From `customer_purchases`, create a description such as **"2 units @ $20.00 = $40.00"**.

Answer

```
SELECT CONCAT(quantity, ' units @ $', unit_customer_price,
              ' = $', quantity * unit_customer_price) AS description
FROM customer_purchases;
```

Example output: `2 units @ $20.00 = $40.00`. CONCAT converts numbers to text automatically.

Q132. Show customer initials in the format **"A.J."**.

Answer

```
SELECT CONCAT(LEFT(customer_first_name, 1), '.', LEFT(customer_last_name, 1), '.') AS initials
FROM customer;
```

Q133. Create a product display string: **"Product 1 [SKU1] - Small"**.

Answer

```
SELECT CONCAT(product_name, ' [', product_sku, '] - ', product_size) AS product_display
FROM product;
```

Q134. What does `SELECT 'Farm' || 'Market';` return in MySQL by default? Why is `||` risky for string joining in MySQL, compared with other databases?

Answer

It returns **0**. In MySQL `||` means **logical OR** (unless the `PIPES_AS_CONCAT` SQL mode is on). Both strings are converted to numbers, 'Farm' becomes 0 and 'Market' becomes 0, and 0 OR 0 gives 0. In Oracle, PostgreSQL and SQLite `||` joins strings, so code ported to MySQL fails silently. MySQL 8 also marks `||` as OR as deprecated. The `market_date table.txt` file in this dataset uses `||` inside CONCAT, which is exactly this trap. Use `CONCAT()` in MySQL.

Q135. Create an `inventory_key` for each `vendor_inventory` row by joining `market_date`, `vendor_id` and `product_id` with underscores (e.g. `2024-01-01_1_1`).

Answer

```
SELECT CONCAT(market_date, '_', vendor_id, '_', product_id) AS inventory_key,
       quantity, original_price
FROM vendor_inventory;
```

Example output: `2024-01-01_1_1`.

Q136. Find the customer whose full name, built with CONCAT inside the WHERE clause, is `'Grace Davis'`. Why can this be slower than filtering on the two columns separately?

Answer

```
SELECT * FROM customer
WHERE CONCAT(customer_first_name, ' ', customer_last_name) = 'Grace Davis';
```

It can be slower because wrapping the columns in a function makes the condition **non-sargable**: MySQL must build the string for every row and can't use an index on the name columns. Better:

```
WHERE customer_first_name = 'Grace' AND customer_last_name = 'Davis'
```

Section 8: INSTR, LEFT, RIGHT, SUBSTRING

Q137. Show the first 3 letters of each customer's first name in uppercase, as `short_code`.

Answer

```
SELECT customer_first_name, UPPER(LEFT(customer_first_name, 3)) AS short_code
FROM customer;
```

Returns ALI, BOB, CHA, ...

Q138. Show the last 2 characters of each customer's zip code.

Answer

```
SELECT customer_zip, RIGHT(customer_zip, 2) AS zip_last2 FROM customer;
```

Q139. From `market_date_info`, show `year_month` (e.g. `2024-01`) using LEFT and `day_of_month` using RIGHT.

Answer

```
SELECT market_date,
       LEFT(market_date, 7)  AS year_month,
       RIGHT(market_date, 2) AS day_of_month
FROM market_date_info;
```

Returns '2024-01' and '01' to '20'. The DATE value is converted to 'YYYY-MM-DD' text first.

Q140. Show each `market_day` as a 3-letter uppercase abbreviation (`MON`, `TUE`, ...).

Answer

```
SELECT market_date, UPPER(LEFT(market_day, 3)) AS day_abbr FROM market_date_info;
```

Q141. Using INSTR, find the position of `'x'` in each `booth_size`.

Answer

```
SELECT booth_number, booth_size, INSTR(booth_size, 'x') AS x_position FROM booth;
```

Returns 3 for '10x10' and 2 for '8x8' and '6x6'.

Q142. Split `booth_size` into `booth_width` and `booth_length`. Don't hard-code positions, because the sizes have different lengths (`'10x10'` vs `'8x8'`).

Answer

```
SELECT booth_number, booth_size,
       LEFT(booth_size, INSTR(booth_size, 'x') - 1)   AS booth_width,
       SUBSTRING(booth_size, INSTR(booth_size, 'x') + 1) AS booth_length
FROM booth;
```

Q143. Using the width and length from Q142, calculate each booth's `area` and its `price_per_sq_ft` (rounded to 2 decimals).

Answer

```
SELECT booth_number, booth_size, booth_price,
       LEFT(booth_size, INSTR(booth_size, 'x') - 1)
         * SUBSTRING(booth_size, INSTR(booth_size, 'x') + 1)          AS area,
       ROUND(booth_price /
         (LEFT(booth_size, INSTR(booth_size, 'x') - 1)
          * SUBSTRING(booth_size, INSTR(booth_size, 'x') + 1)), 2)    AS price_per_sq_ft
FROM booth;
```

The areas are 100, 64 and 36. Examples: booth 1 costs 1.00 per sq ft, booth 2 costs 1.64 and booth 3 costs 3.06. The strings are converted to numbers automatically when multiplied.

Q144. Extract the number from `product_name` (e.g. `'Product 12'` → `12`), using INSTR to locate the space.

Answer

```
SELECT product_name,
       SUBSTRING(product_name, INSTR(product_name, ' ') + 1) + 0 AS product_number
FROM product;
```

`+ 0` turns the text '12' into the number 12, which matters when sorting.

Q145. Extract the number from `product_sku` (text after `'SKU'`) and return only the rows where that number does **not** match `product_id`.

Answer

```
SELECT product_id, product_sku,
       SUBSTRING(product_sku, LENGTH('SKU') + 1) AS sku_number
FROM product
WHERE SUBSTRING(product_sku, LENGTH('SKU') + 1) + 0 <> product_id;
```

Returns no rows: every SKU number matches its product id.

Q146. From `vendor`, extract the vendor number from `vendor_name` and the type letter (`A` / `B`) from `vendor_type`.

Answer

```
SELECT vendor_name, vendor_type,
       SUBSTRING(vendor_name, INSTR(vendor_name, ' ') + 1) AS vendor_number,
       RIGHT(vendor_type, 1)                               AS type_letter
FROM vendor;
```

Q147. Extract the first word of `booth_description` (e.g. `'Large'`), using INSTR.

Answer

```
SELECT booth_description,
       LEFT(booth_description, INSTR(booth_description, ' ') - 1) AS first_word
FROM booth;
```

Q148. Using INSTR (not LIKE), find the customers whose first name does **not** contain the letter `'e'`.

Answer

```
SELECT * FROM customer WHERE INSTR(customer_first_name, 'e') = 0;
```

Returns 13 customers. INSTR returns **0** when the text isn't found, and it follows the collation, so 'Eve' counts as containing 'e'.

Q149. Find the customers whose last name contains `'a'` somewhere **after** the 3rd position.

Answer

```
SELECT customer_last_name FROM customer
WHERE INSTR(SUBSTRING(customer_last_name, 4), 'a') > 0;
```

Returns Thomas ('a' at position 5). `INSTR(customer_last_name, 'a') > 3` also works for this data. However, it only checks the **first** 'a', so a name like 'Taylara' would be missed.

Q150. Compare `SUBSTRING(customer_last_name, -3)` with `RIGHT(customer_last_name, 3)`. Are the results the same?

Answer

In this data they are the same, because every last name has at least 3 characters (Lee has exactly 3). The difference appears when the string is **shorter** than n: `RIGHT('Li', 3)` returns 'Li', but `SUBSTRING('Li', -3)` returns '' (empty), because the start position falls before the beginning of the string.

Q151. From `customer_purchases`, extract the month from `market_date` using SUBSTRING, and the hour and minute from `transaction_time` as separate columns.

Answer

```
SELECT market_date,
       SUBSTRING(market_date, 6, 2)      AS month_part,
       LEFT(transaction_time, 2)         AS hour_part,
       SUBSTRING(transaction_time, 4, 2) AS minute_part
FROM customer_purchases;
```

Q152. Show the 2nd to 4th letters of each customer's first name.

Answer

```
SELECT customer_first_name, SUBSTRING(customer_first_name, 2, 3) AS letters_2_to_4
FROM customer;
```

Examples: Alice → 'lic'; Bob → 'ob' (only 2 characters are left).

Q153. Find the customers whose first name starts with a letter that comes **later** in the alphabet than the first letter of their last name (e.g. Tom Robinson: T > R).

Answer

```
SELECT customer_first_name, customer_last_name FROM customer
WHERE LEFT(customer_first_name, 1) > LEFT(customer_last_name, 1);
```

Returns 11 customers: Charlie, Grace, Jack, Leo, Nick, Olive, Paul, Quincy, Rita, Steve and Tom.

Q154. Mask customer zip codes so that only the first 3 digits show: `'12345'` → `'123**'`.

Answer

```
SELECT customer_zip, CONCAT(LEFT(customer_zip, 3), '**') AS masked_zip FROM customer;
```

Q155. Mask each customer's last name so that only the first letter shows and the rest are `*` (e.g. `Johnson` → `J******`). The number of stars must match the name length.

Answer

```
SELECT customer_last_name,
       CONCAT(LEFT(customer_last_name, 1),
              REPEAT('*', LENGTH(customer_last_name) - 1)) AS masked_last_name
FROM customer;
```

Q156. What do these return in MySQL? Explain any surprising results.

```
SELECT SUBSTRING('Farmers', 1, 3), SUBSTRING('Farmers', 0, 3), SUBSTRING('Farmers', 4),
SUBSTRING('Farmers', -4, 2);
```

Answer

Expression	Result	Why
`SUBSTRING('Farmers', 1, 3)`	`'Far'`	Positions start at 1
`SUBSTRING('Farmers', 0, 3)`	`''`	Position 0 doesn't exist in MySQL, so the result is empty
`SUBSTRING('Farmers', 4)`	`'mers'`	From position 4 to the end
`SUBSTRING('Farmers', -4, 2)`	`'me'`	A negative start counts from the end (-4 is 'm'), then takes 2 characters

Q157. INSTR only finds the first occurrence. Find the position of the **second** `'o'` in each customer's last name, and show only names that have a second `'o'`.

Answer

```
SELECT customer_last_name,
       INSTR(customer_last_name, 'o')
       + INSTR(SUBSTRING(customer_last_name, INSTR(customer_last_name, 'o') + 1), 'o')
         AS second_o_position
FROM customer
WHERE INSTR(customer_last_name, 'o') > 0
  AND INSTR(SUBSTRING(customer_last_name, INSTR(customer_last_name, 'o') + 1), 'o') > 0;
```

Returns Johnson (6), Moore (3) and Robinson (7). The approach: find the first 'o', cut the string just after it, search again, and add the two positions.

Q158. From `market_date_info`, extract the year from `market_date` using LEFT, and return any rows where it does not match the `market_year` column.

Answer

```
SELECT market_date, market_year, LEFT(market_date, 4) AS year_from_date
FROM market_date_info
WHERE LEFT(market_date, 4) <> market_year;
```

Returns no rows, so the data is consistent.

Q159. Using RIGHT instead of LIKE, find the vendors whose owner's last name ends with `'son'`.

Answer

```
SELECT * FROM vendor WHERE RIGHT(vendor_owner_last_name, 3) = 'son';
```

Returns Carol Johnson, Hank Wilson, Kathy Anderson and Mona Jackson.

Q160. Create a product code of the form `S-1`, `M-2`, `L-3`: the first letter of `product_size`, a hyphen, then `product_id`.

Answer

```
SELECT product_id, CONCAT(LEFT(product_size, 1), '-', product_id) AS product_code
FROM product;
```

Q161. Find the transactions whose `transaction_time` is exactly on the hour (minutes and seconds are `00:00`), using RIGHT.

Answer

```
SELECT * FROM customer_purchases WHERE RIGHT(transaction_time, 5) = '00:00';
```

Returns all 20 rows.

Q162. Using SUBSTRING and INSTR (no hard-coded positions), extract the text that comes after `'Type '` in `vendor_type`.

Answer

```
SELECT vendor_type,
       SUBSTRING(vendor_type, INSTR(vendor_type, ' ') + 1) AS type_code
FROM vendor;
```

Q163. Find the customers whose zip code has `'5'` as its **third** digit.

Answer

```
SELECT * FROM customer WHERE SUBSTRING(customer_zip, 3, 1) = '5';
```

Returns Charlie and Mona (34567).

Q164. From `vendor`, show the owner's initials (e.g. `JD`) and return only the vendors where **both** initials fall in the first half of the alphabet (A–M).

Answer

```
SELECT vendor_id,
       CONCAT(LEFT(vendor_owner_first_name, 1), LEFT(vendor_owner_last_name, 1)) AS initials
FROM vendor
WHERE LEFT(vendor_owner_first_name, 1) BETWEEN 'A' AND 'M'
  AND LEFT(vendor_owner_last_name, 1)  BETWEEN 'A' AND 'M';
```

Returns 9 vendors: JD, AB, CJ, EJ, FD, GM, IM, KA and MJ. BETWEEN works here because each value is exactly one character.

Section 9: TRIM, LENGTH, REVERSE

Q165. Show each customer's full name and its length (including the space), longest first.

Answer

```
SELECT CONCAT(customer_first_name, ' ', customer_last_name)         AS full_name,
       LENGTH(CONCAT(customer_first_name, ' ', customer_last_name)) AS name_length
FROM customer
ORDER BY name_length DESC;
```

The longest names are Alice Johnson, Charlie Brown and Jack Anderson (13 characters).

Q166. Find the customers whose first and last names have the same number of characters.

Answer

```
SELECT * FROM customer
WHERE LENGTH(customer_first_name) = LENGTH(customer_last_name);
```

Returns Grace Davis and Steve Lewis.

Q167. What will this return?

```
SELECT LENGTH('  Vendor 1  '), LENGTH(TRIM('  Vendor 1  ')),
       LENGTH(LTRIM('  Vendor 1  ')), LENGTH(RTRIM('  Vendor 1  '));
```

Answer

Expression	Result
`LENGTH(' Vendor 1 ')`	**12**
`LENGTH(TRIM(...))`	**8**
`LENGTH(LTRIM(...))`	**10**
`LENGTH(RTRIM(...))`	**10**

Q168. Use `TRIM(LEADING '0' FROM customer_zip)` and return only the customers whose zip value changes as a result.

Answer

```
SELECT customer_zip, TRIM(LEADING '0' FROM customer_zip) AS trimmed_zip
FROM customer
WHERE TRIM(LEADING '0' FROM customer_zip) <> customer_zip;
```

Returns Jack and Tom ('01234' becomes '1234').

Q169. Use `TRIM(TRAILING ' booth' FROM booth_description)` to get just the size word (`Large`, `Medium`, `Small`).

Answer

```
SELECT booth_description,
       TRIM(TRAILING ' booth' FROM booth_description) AS size_word
FROM booth;
```

Q170. Using TRIM with LEADING (not SUBSTRING), extract the numeric part of `product_sku` and of `product_name`.

Answer

```
SELECT product_sku,  TRIM(LEADING 'SKU'      FROM product_sku)  AS sku_number,
       product_name, TRIM(LEADING 'Product ' FROM product_name) AS name_number
FROM product;
```

TRIM removes a whole **string** (here 'SKU'), not individual characters.

Q171. What will `TRIM(TRAILING 'day' FROM market_day)` return for each weekday? Pay attention to Tuesday, Wednesday and Thursday.

Answer

market_day	Result
Monday	Mon
Tuesday	**Tues**
Wednesday	**Wednes**
Thursday	**Thurs**
Friday	Fri
Saturday	**Satur**
Sunday	Sun

TRIM removes only the suffix 'day'. It doesn't produce standard 3-letter abbreviations; use `LEFT(market_day, 3)` for those.

Q172. What is the difference between `LENGTH('Café')` and `CHAR_LENGTH('Café')` in MySQL? Why does it matter for names with accented characters?

Answer

`LENGTH` counts **bytes** and `CHAR_LENGTH` counts **characters**. In utf8mb4, 'é' takes 2 bytes, so `LENGTH('Café')` = **5** and `CHAR_LENGTH('Café')` = **4**. Using LENGTH for validation ("the name must be at most 10 characters") or with LEFT/SUBSTRING arithmetic gives wrong results for accented or non-Latin names.

Q173. Using REVERSE, find the customers and the vendor owners whose first name is a palindrome (reads the same backwards, ignoring case). Write one query per table.

Answer

```
SELECT customer_first_name FROM customer
WHERE LOWER(customer_first_name) = LOWER(REVERSE(customer_first_name));

SELECT vendor_owner_first_name FROM vendor
WHERE LOWER(vendor_owner_first_name) = LOWER(REVERSE(vendor_owner_first_name));
```

Both queries return **Bob** and **Eve**. `REVERSE('Bob')` is 'boB', so LOWER (or the case-insensitive collation) is needed to match.

Q174. Extract the **last word** of `product_name` using REVERSE and INSTR, without assuming the word's length.

Answer

```
SELECT product_name,
       RIGHT(product_name, INSTR(REVERSE(product_name), ' ') - 1) AS last_word
FROM product;
```

Reversing the string makes the **last** space the **first** one. Its position minus 1 is the length of the last word.

Q175. Find the position of the **last** `'o'` in each customer's last name, using REVERSE, INSTR and LENGTH.

Answer

```
SELECT customer_last_name,
       CASE WHEN INSTR(customer_last_name, 'o') = 0 THEN 0
            ELSE LENGTH(customer_last_name) - INSTR(REVERSE(customer_last_name), 'o') + 1
       END AS last_o_position
FROM customer;
```

Examples: Johnson → 6, Moore → 3, Robinson → 7. The CASE handles names without an 'o'. Without it, the formula would return LENGTH + 1.

Q176. Find the customers whose last name is at least 3 characters longer than their first name.

Answer

```
SELECT * FROM customer
WHERE LENGTH(customer_last_name) - LENGTH(customer_first_name) >= 3;
```

Returns Ivy Taylor, Jack Anderson, Leo Jackson and Tom Robinson.

Q177. Count how many times the letter `'a'` (upper or lower case) appears in each customer's full name, using LENGTH and REPLACE. Show only customers with 2 or more.

Answer

```
SELECT CONCAT(customer_first_name, ' ', customer_last_name) AS full_name,
       LENGTH(CONCAT(customer_first_name, customer_last_name))
       - LENGTH(REPLACE(LOWER(CONCAT(customer_first_name, customer_last_name)), 'a', '')) AS
a_count
FROM customer
WHERE LENGTH(CONCAT(customer_first_name, customer_last_name))
      - LENGTH(REPLACE(LOWER(CONCAT(customer_first_name, customer_last_name)), 'a', '')) >= 2;
```

Returns Grace Davis, Jack Anderson, Kathy Thomas and Rita Clark (2 each). **REPLACE is case-sensitive** even under a `_ci` collation, which is why LOWER is needed to catch the 'A' in Anderson.

Q178. Count the words in each `booth_description`, using LENGTH and REPLACE.

Answer

```
SELECT booth_description,
       LENGTH(booth_description) - LENGTH(REPLACE(booth_description, ' ', '')) + 1 AS
word_count
FROM booth;
```

Returns 2 for every booth. Number of words = number of spaces + 1, assuming single spaces between words.

Q179. Using LENGTH instead of LIKE, find the vendors whose name has a two-digit number (e.g. `'Vendor 10'`).

Answer

```
SELECT * FROM vendor WHERE LENGTH(vendor_name) = 9;
```

Returns Vendor 10 to Vendor 20 (11 rows).

Q180. What will `SELECT TRIM(BOTH '0' FROM '00120400');` return?

Answer

It returns **'1204'**. Only the leading and trailing '0' characters are removed; the zero inside stays.

Q181. What does `REVERSE(booth_price)` return for a booth priced at 100.00? What does this tell you about how REVERSE handles numbers?

Answer

It returns the string **'00.001'**. REVERSE converts the number to its text form ('100.00') and reverses the characters. The result is always a string, not a number.

Q182. Using REVERSE and LEFT only (no RIGHT, no LIKE), find the customers whose first name ends with `'e'`.

Answer

```
SELECT * FROM customer WHERE LEFT(REVERSE(customer_first_name), 1) = 'e';
```

Returns Alice, Charlie, Eve, Grace, Olive and Steve.

Section 10: ROUND & Numeric Formatting

Q183. From `market_date_info`, convert the average temperature (midpoint of min and max) to Celsius and show it rounded to 0 decimals and to 1 decimal, side by side.

Answer

```
SELECT market_date,
       ROUND(((market_temp_min + market_temp_max) / 2 - 32) * 5 / 9, 0) AS avg_c_0dp,
       ROUND(((market_temp_min + market_temp_max) / 2 - 32) * 5 / 9, 1) AS avg_c_1dp
FROM market_date_info;
```

Example: 2024-01-01 has an average of 37.5 °F, which gives 3 and 3.1.

Q184. Round each `booth_price` to the nearest 10 and to the nearest 100. What happens for prices of 145 and 155?

Answer

```
SELECT booth_number, booth_price,
       ROUND(booth_price, -1) AS nearest_10,
       ROUND(booth_price, -2) AS nearest_100
FROM booth;
```

145 becomes 150 and 100; 155 becomes 160 and 200. A negative second argument rounds to the left of the decimal point, and exact values round half away from zero.

Q185. What will this return in MySQL?

```
SELECT ROUND(2.5), ROUND(3.5), ROUND(-2.5), ROUND(2.45, 1), ROUND(1234.5678, -2);
```

Answer

Expression	Result
`ROUND(2.5)`	**3**
`ROUND(3.5)`	**4**
`ROUND(-2.5)`	**-3**
`ROUND(2.45, 1)`	**2.5**
`ROUND(1234.5678, -2)`	**1200**

For exact (DECIMAL) values MySQL rounds half **away from zero**; it does not use banker's rounding. Floating-point values (DOUBLE) can behave differently.

Q186. From `customer_purchases`, calculate the total with 7.25% tax, both rounded and truncated to 2 decimals (`ROUND` vs `TRUNCATE`). Show a column with the difference between the two.

Answer

```
SELECT customer_id,
       quantity * unit_customer_price * 1.0725                    AS raw_total,
       ROUND(quantity * unit_customer_price * 1.0725, 2)          AS rounded_total,
       TRUNCATE(quantity * unit_customer_price * 1.0725, 2)       AS truncated_total,
       ROUND(quantity * unit_customer_price * 1.0725, 2)
         - TRUNCATE(quantity * unit_customer_price * 1.0725, 2)   AS difference
FROM customer_purchases;
```

Example: 63 × 1.0725 = 67.5675, which rounds to 67.57 and truncates to 67.56 (a difference of 0.01).

Q187. From `vendor_inventory`, apply a 12.5% discount to `original_price` and round the result to the **nearest 5 cents** (0.05).

Answer

```
SELECT original_price,
       original_price * 0.875                  AS discounted,
       ROUND(original_price * 0.875 * 20) / 20 AS nearest_5_cents
FROM vendor_inventory;
```

Example: 15 × 0.875 = 13.125. Multiplied by 20 that is 262.5, which rounds to 263, and 263 ÷ 20 = 13.15. Rounding to the nearest step s is `ROUND(x / s) * s` .

Q188. Without using CEIL/CEILING, round the discounted price from Q187 **up** to the next whole dollar.

Answer

```
SELECT original_price * 0.875 AS discounted,
       TRUNCATE(original_price * 0.875, 0)
       + (original_price * 0.875 > TRUNCATE(original_price * 0.875, 0)) AS rounded_up
FROM vendor_inventory;
```

Example: 13.125 becomes 14, while 14.000 stays 14. TRUNCATE removes the decimal part, and the comparison adds 1 (TRUE) only when there *was* a decimal part.

Q189. Show each booth's price as a percentage of the top price (195). Round to 1 decimal and add a `%` sign, e.g. `51.3%`.

Answer

```
SELECT booth_number, booth_price,
       CONCAT(ROUND(booth_price * 100 / 195, 1), '%') AS pct_of_top
FROM booth;
```

Examples: 100 → `51.3%`; 195 → `100.0%`.

Q190. From `customer_purchases`, apply three successive 10% discounts to `unit_customer_price` and round to 2 decimals. Is that the same as a single 30% discount?

Answer

```
SELECT unit_customer_price,
       ROUND(unit_customer_price * 0.9 * 0.9 * 0.9, 2) AS three_discounts,
       ROUND(unit_customer_price * 0.7, 2)             AS single_30pct
FROM customer_purchases;
```

They are **not** the same: $0.9^3 = 0.729$, which is a 27.1% discount. For 20.00 the results are 14.58 and 14.00.

Q191. Find the purchases whose total, including 8% tax and rounded to the nearest dollar, is more than 500.

Answer

```
SELECT *, ROUND(quantity * unit_customer_price * 1.08) AS total_with_tax
FROM customer_purchases
WHERE ROUND(quantity * unit_customer_price * 1.08) > 500;
```

Returns 6 rows (customers 14–19). Customer 14: 495 × 1.08 = 534.6, which rounds to 535.

Q192. What will `SELECT ROUND('12.567', 1), ROUND('12.567abc', 1);` return? Explain.

Answer

- `ROUND('12.567', 1)` returns **12.6**. The string is converted to a number (DOUBLE) first.

- `ROUND('12.567abc', 1)` also returns **12.6**, with a warning ("Truncated incorrect DOUBLE value"). MySQL reads the leading numeric part and ignores the rest.

Silent conversions like this hide dirty data.

Q193. From `market_date_info`, show the temperature range as a percentage of the maximum temperature, rounded to 1 decimal.

Answer

```
SELECT market_date,
       ROUND((market_temp_max - market_temp_min) * 100 / market_temp_max, 1) AS
range_pct_of_max
FROM market_date_info;
```

Example: 2024-01-01 gives 15 × 100 / 45 = 33.3.

Q194. A vendor packs produce in boxes of 12. From `vendor_inventory`, show `ROUND(quantity / 12)` as boxes needed. Why is ROUND the wrong function here? Give a correct expression.

Answer

ROUND rounds **down** when the fraction is below .5, so some units are left without a box. For 40 units, 40 / 12 = 3.33, which rounds to 3 boxes (only 36 units packed). Packing needs a ceiling:

```
SELECT quantity,
       ROUND(quantity / 12) AS wrong_boxes,
       (quantity + 11) DIV 12 AS boxes_needed      -- or CEIL(quantity / 12)
FROM vendor_inventory;
```

Section 11: DISTINCT

Q195. List the distinct booth sizes.

Answer

```
SELECT DISTINCT booth_size FROM booth;
```

Returns '10x10', '8x8' and '6x6'.

Q196. List the distinct `(booth_description, booth_size)` pairs. What does the result tell you about how consistent the booth data is?

Answer

```
SELECT DISTINCT booth_description, booth_size FROM booth;
```

Returns exactly 3 pairs (Large/10x10, Medium/8x8, Small/6x6), so each description always goes with the same size and the data is consistent. A 4th pair would reveal a mismatch.

Q197. List the distinct customer zip codes in ascending order. Why does `'01234'` appear first?

Answer

```
SELECT DISTINCT customer_zip FROM customer ORDER BY customer_zip;
```

Returns 10 zips. '01234' comes first because the column is text, and the character '0' sorts before '1'.

Q198. Explain what `SELECT DISTINCT market_season, market_day FROM market_date_info;` returns. Does DISTINCT apply to one column or to both?

Answer

DISTINCT applies to the **whole row** (the combination of all selected columns), not only to the first column. Every date is Winter, so it returns the 7 unique (Winter, day) pairs.

Q199. List the distinct first letters of customer last names, sorted alphabetically.

Answer

```
SELECT DISTINCT LEFT(customer_last_name, 1) AS first_letter
FROM customer
ORDER BY first_letter;
```

Returns 13 letters: A, B, C, D, H, J, L, M, P, R, S, T and W.

Q200. List the distinct year-month values (`'2024-01'`) from `market_date_info`, using LEFT.

Answer

```
SELECT DISTINCT LEFT(market_date, 7) AS year_month FROM market_date_info;
```

Returns one row: '2024-01'.

Q201. List the distinct vendor owner last names. Which last name(s) appear more than once in the table? (Find out by comparing with the full list; don't use GROUP BY.)

Answer

```
SELECT DISTINCT vendor_owner_last_name FROM vendor ORDER BY vendor_owner_last_name;
```

Returns 19 names from 20 rows. **White** appears twice (Bob White, vendor 4, and Nick White, vendor 16).

Q202. Does `SELECT DISTINCT(vendor_type), vendor_name FROM vendor;` return only distinct vendor types? Explain what the parentheses actually do.

Answer

No. DISTINCT is not a function. The brackets only group the expression `(vendor_type)`, and DISTINCT still applies to the whole row (vendor_type, vendor_name). Every vendor name is unique, so all **20 rows** are returned.

Q203. List the distinct product sizes, excluding 'Small'.

Answer

```
SELECT DISTINCT product_size FROM product WHERE product_size <> 'Small';
```

Returns Medium and Large.

Q204. Using CASE, label each market date as 'Snow', 'Rain' or 'Dry', and list the distinct labels.

Answer

```
SELECT DISTINCT
       CASE WHEN market_snow_flag = 1 THEN 'Snow'
            WHEN market_rain_flag = 1 THEN 'Rain'
            ELSE 'Dry' END AS weather_label
FROM market_date_info;
```

Returns 'Dry' and 'Snow'. 'Rain' never appears, because the data has no rain days.

Q205. If a column contains three NULL values, how many NULL rows will `SELECT DISTINCT` on that column return?

Answer

One. DISTINCT treats all NULLs as duplicates of each other, even though `NULL = NULL` is not TRUE.

Q206. Does this query run in MySQL 8? Explain the error or the result.

```
SELECT DISTINCT market_day FROM market_date_info ORDER BY market_date;
```

Answer

It **fails** with error 3065: "Expression #1 of ORDER BY clause is not in SELECT list ... this is incompatible with DISTINCT". After DISTINCT collapses the rows, each `market_day` value (for example 'Monday') matches several `market_date` values, so MySQL can't know which one to sort by. Fix: sort by a selected column (`ORDER BY market_day`), or add `market_date` to the select list.

Q207. List the distinct combinations of `market_week` and `market_day` on which it snowed.

Answer

```
SELECT DISTINCT market_week, market_day
FROM market_date_info
WHERE market_snow_flag = 1;
```

Returns 5 rows: (1, Wednesday), (1, Friday), (2, Wednesday), (3, Wednesday) and (3, Saturday).

Q208. List the distinct temperature ranges (max − min) from `market_date_info`, sorted from smallest to largest.

Answer

```
SELECT DISTINCT market_temp_max - market_temp_min AS temp_range
FROM market_date_info
ORDER BY temp_range;
```

Returns 13, 14, 15 and 16.

Section 12: CASE Statements

Q209. Classify booths by price: `'Premium'` (≥ 175), `'Standard'` (130 to 174.99), `'Economy'` (below 130).

Answer

```
SELECT booth_number, booth_price,
       CASE WHEN booth_price >= 175 THEN 'Premium'
            WHEN booth_price >= 130 THEN 'Standard'
            ELSE 'Economy' END AS price_category
FROM booth;
```

Returns Premium for booths 16–20, Standard for booths 7–15 and Economy for booths 1–6.

Q210. Label each market date as `'Weekend'` or `'Weekday'`.

Answer

```
SELECT market_date, market_day,
       CASE WHEN market_day IN ('Saturday', 'Sunday') THEN 'Weekend'
            ELSE 'Weekday' END AS day_type
FROM market_date_info;
```

Q211. Label each market date as `'Snow Day'`, `'Rain Day'` or `'Clear'`. If a date ever had both flags set to 1, which label would it get? Why does the order of WHEN clauses matter?

Answer

```
SELECT market_date,
       CASE WHEN market_snow_flag = 1 THEN 'Snow Day'
            WHEN market_rain_flag = 1 THEN 'Rain Day'
            ELSE 'Clear' END AS weather
FROM market_date_info;
```

A date with both flags set would get **'Snow Day'**. CASE returns the **first** WHEN that is true and skips the rest. If both conditions should show, add a WHEN for `snow = 1 AND rain = 1` at the top.

Q212. Put each purchase into a bill tier: `'High'` (total > 500), `'Medium'` (200 to 500), `'Low'` (below 200).

Answer

```
SELECT customer_id, quantity * unit_customer_price AS total_amount,
       CASE WHEN quantity * unit_customer_price > 500  THEN 'High'
            WHEN quantity * unit_customer_price >= 200 THEN 'Medium'
            ELSE 'Low' END AS bill_tier
FROM customer_purchases;
```

Returns 5 High, 8 Medium and 7 Low bills.

Q213. What will this return for a booth priced at 170? Fix the logic.

```
SELECT booth_price,
       CASE WHEN booth_price > 100 THEN 'Mid'
            WHEN booth_price > 150 THEN 'High'
            ELSE 'Low' END AS tier
FROM booth;
```

Answer

A booth priced at 170 gets **'Mid'**. It matches `> 100` first, so the `> 150` branch is never reached, and no booth can ever be 'High'. Put the most restrictive condition first:

```
CASE WHEN booth_price > 150 THEN 'High'
     WHEN booth_price > 100 THEN 'Mid'
     ELSE 'Low' END
```

Q214. Write a CASE on `booth` without an ELSE clause that only labels Large booths. What value do the other booths get?

Answer

```
SELECT booth_number,
       CASE WHEN booth_size = '10x10' THEN 'Large' END AS large_flag
FROM booth;
```

The other booths get **NULL**. A CASE with no matching WHEN and no ELSE returns NULL.

Q215. Sort the products by size in the order **Small** → **Medium** → **Large** (not alphabetical), using CASE in ORDER BY. Break ties by product_id.

Answer

```
SELECT * FROM product
ORDER BY CASE product_size WHEN 'Small'  THEN 1
                           WHEN 'Medium' THEN 2
                           WHEN 'Large'  THEN 3 END,
         product_id;
```

Q216. Using the first digit of `customer_zip`, assign a region: 0–3 → `'East'`, 4–6 → `'Central'`, 7–9 → `'West'`.

Answer

```
SELECT customer_first_name, customer_zip,
       CASE WHEN LEFT(customer_zip, 1) BETWEEN '0' AND '3' THEN 'East'
            WHEN LEFT(customer_zip, 1) BETWEEN '4' AND '6' THEN 'Central'
            ELSE 'West' END AS region
FROM customer;
```

Returns 8 East, 6 Central and 6 West customers.

Q217. Classify each purchase by `transaction_time` into `'Morning'` (06:00–11:59), `'Afternoon'` (12:00–16:59), `'Evening'` (17:00–21:59) and `'Night'` (everything else).

Answer

```
SELECT customer_id, transaction_time,
       CASE WHEN transaction_time BETWEEN '06:00:00' AND '11:59:59' THEN 'Morning'
            WHEN transaction_time BETWEEN '12:00:00' AND '16:59:59' THEN 'Afternoon'
            WHEN transaction_time BETWEEN '17:00:00' AND '21:59:59' THEN 'Evening'
            ELSE 'Night' END AS shift
FROM customer_purchases;
```

Returns 2 Morning, 5 Afternoon, 5 Evening and 8 Night purchases. The Night shift crosses midnight, so ELSE catches it.

Q218. Convert `transaction_time` into a 12-hour label such as `'10 AM'`, `'12 PM'`, `'1 PM'`, `'12 AM'`, using CASE with LEFT and arithmetic.

Answer

```
SELECT transaction_time,
       CASE WHEN LEFT(transaction_time, 2) + 0 = 0  THEN '12 AM'
            WHEN LEFT(transaction_time, 2) + 0 < 12 THEN CONCAT(LEFT(transaction_time, 2) + 0,
' AM')
            WHEN LEFT(transaction_time, 2) + 0 = 12 THEN '12 PM'
            ELSE CONCAT(LEFT(transaction_time, 2) - 12, ' PM') END AS time_12h
FROM customer_purchases;
```

`+ 0` turns '05' into 5 and drops the leading zero. Midnight and noon need their own branches.

Q219. Write a searched CASE that maps `market_day` to `'Start of Week'` (Monday, Tuesday), `'Mid Week'` (Wednesday, Thursday) and `'Week End'` (the rest). Then rewrite it as a simple CASE. When is each form better?

Answer

```
-- Searched CASE
CASE WHEN market_day IN ('Monday', 'Tuesday')    THEN 'Start of Week'
     WHEN market_day IN ('Wednesday', 'Thursday') THEN 'Mid Week'
     ELSE 'Week End' END

-- Simple CASE
CASE market_day WHEN 'Monday'    THEN 'Start of Week'
                WHEN 'Tuesday'   THEN 'Start of Week'
                WHEN 'Wednesday' THEN 'Mid Week'
                WHEN 'Thursday'  THEN 'Mid Week'
                ELSE 'Week End' END
```

A simple CASE compares **one expression for equality** against each value, so it is neat for exact lookups. A searched CASE takes any condition (ranges, IN, LIKE, several columns, IS NULL), so it is more flexible. A simple CASE can't test `WHEN NULL`.

Q220. Data check: flag each booth `'OK'` if its description matches its size (Large ↔ 10x10, Medium ↔ 8x8, Small ↔ 6x6), otherwise `'Mismatch'`.

Answer

```
SELECT booth_number, booth_description, booth_size,
       CASE WHEN (booth_description = 'Large booth'  AND booth_size = '10x10')
              OR (booth_description = 'Medium booth' AND booth_size = '8x8')
              OR (booth_description = 'Small booth'  AND booth_size = '6x6')
            THEN 'OK' ELSE 'Mismatch' END AS consistency
FROM booth;
```

All 20 booths are 'OK'.

Q221. Product sizes should follow the cycle Small, Medium, Large by `product_id` (1 → Small, 2 → Medium, 3 → Large, 4 → Small, ...). Use CASE with `%` to work out the expected size, and return only the products whose actual size is different.

Answer

```
SELECT product_id, product_size,
       CASE product_id % 3 WHEN 1 THEN 'Small'
                           WHEN 2 THEN 'Medium'
                           ELSE 'Large' END AS expected_size
FROM product
WHERE product_size <> CASE product_id % 3 WHEN 1 THEN 'Small'
                                          WHEN 2 THEN 'Medium'
                                          ELSE 'Large' END;
```

Returns **product 20**: it should be Medium, but it is stored as Large.

Q222. From `vendor_inventory`, add a `stock_status` column (`'Low'` < 50, `'Moderate'` 50–149, `'High'` ≥ 150) and a `restock_needed` column (`'Yes'` or `'No'`).

Answer

```
SELECT vendor_id, quantity,
       CASE WHEN quantity < 50  THEN 'Low'
            WHEN quantity < 150 THEN 'Moderate'
            ELSE 'High' END AS stock_status,
       CASE WHEN quantity < 50 THEN 'Yes' ELSE 'No' END AS restock_needed
FROM vendor_inventory;
```

Returns Low for vendors 1–4, Moderate for 5–14 and High for 15–20.

Q223. Label each market date's feel with a nested CASE: if it snowed, `'Snowy & Cold'` when min < 30, otherwise `'Snowy'`. If it didn't snow, `'Cold'` when min < 30, otherwise `'Mild'`.

Answer

```
SELECT market_date, market_temp_min, market_snow_flag,
       CASE WHEN market_snow_flag = 1 THEN
                 CASE WHEN market_temp_min < 30 THEN 'Snowy & Cold' ELSE 'Snowy' END
            ELSE
                 CASE WHEN market_temp_min < 30 THEN 'Cold' ELSE 'Mild' END
       END AS day_feel
FROM market_date_info;
```

Returns Snowy & Cold for 01-03, 01-05, 01-10 and 01-17; Snowy for 01-20; Cold for 01-12 and 01-18; and Mild for the rest.

Q224. Customers who buy 10 or more units get a 10% discount. Return only the purchases whose **final** amount (after the discount, if any) is above 300. Use CASE inside the WHERE clause.

Answer

```
SELECT customer_id, quantity, quantity * unit_customer_price AS total_amount
FROM customer_purchases
WHERE CASE WHEN quantity >= 10 THEN quantity * unit_customer_price * 0.9
           ELSE quantity * unit_customer_price END > 300;
```

Returns 9 rows (customers 11–19). Customer 10's total of 319 drops to 287.10 after the discount.

Q225. Apply quantity slabs to `customer_purchases`: 1–5 units no discount, 6–10 units 5%, 11–15 units 10%, 16+ units 15%. Show the discount %, discount amount and final amount, rounded to 2 decimals.

Answer

```
SELECT customer_id, quantity, quantity * unit_customer_price AS total_amount,
       CASE WHEN quantity <= 5  THEN 0
            WHEN quantity <= 10 THEN 5
            WHEN quantity <= 15 THEN 10
            ELSE 15 END AS discount_pct,
       ROUND(quantity * unit_customer_price *
             CASE WHEN quantity <= 5  THEN 0
                  WHEN quantity <= 10 THEN 0.05
                  WHEN quantity <= 15 THEN 0.10
                  ELSE 0.15 END, 2) AS discount_amount,
       ROUND(quantity * unit_customer_price *
             (1 - CASE WHEN quantity <= 5  THEN 0
                       WHEN quantity <= 10 THEN 0.05
                       WHEN quantity <= 15 THEN 0.10
                       ELSE 0.15 END), 2) AS final_amount
FROM customer_purchases;
```

Q226. Format customer names with a CASE: if the first name has 3 letters or fewer, show the full name. Otherwise show the first initial and the last name (e.g. `A. Johnson`).

Answer

```
SELECT CASE WHEN LENGTH(customer_first_name) <= 3
            THEN CONCAT(customer_first_name, ' ', customer_last_name)
            ELSE CONCAT(LEFT(customer_first_name, 1), '. ', customer_last_name)
       END AS display_name
FROM customer;
```

Examples: 'Bob Smith' and 'A. Johnson'.

Q227. Map `vendor_type` with a simple CASE: `'Type A'` → `'Organic'`, `'Type B'` → `'Conventional'`, anything else → `'Unclassified'`.

Answer

```
SELECT vendor_name, vendor_type,
       CASE vendor_type WHEN 'Type A' THEN 'Organic'
                        WHEN 'Type B' THEN 'Conventional'
                        ELSE 'Unclassified' END AS vendor_category
FROM vendor;
```

Q228. What happens when the branches of a CASE return different data types, e.g. `CASE WHEN booth_price > 150 THEN 'High' ELSE 0 END`? What type does the result column get?

Answer

MySQL works out **one** result type for the whole CASE. When the branches mix a string and a number, the result type is a **string** (VARCHAR), so the 0 comes back as the text '0'. This affects sorting (text order) and any client code that expects numbers. Keep all branches the same type, for example `ELSE 'Low'`, or `ELSE NULL`.

Q229. Write a nested CASE on `booth`: first split by `booth_size`, then within each size label the booth `'Costly'` or `'Affordable'` depending on whether its price is above 150.

Answer

```
SELECT booth_number, booth_size, booth_price,
       CASE booth_size
            WHEN '10x10' THEN CASE WHEN booth_price > 150 THEN 'Large - Costly'  ELSE 'Large -
Affordable'  END
            WHEN '8x8'   THEN CASE WHEN booth_price > 150 THEN 'Medium - Costly' ELSE 'Medium -
Affordable' END
            ELSE              CASE WHEN booth_price > 150 THEN 'Small - Costly'  ELSE 'Small -
Affordable'  END
       END AS booth_label
FROM booth;
```

Q230. Using CASE with IN, label market days as `'Peak'` (Friday, Saturday, Sunday) or `'Off-Peak'`, and show only Peak days that had no snow.

Answer

```
SELECT market_date, market_day,
       CASE WHEN market_day IN ('Friday', 'Saturday', 'Sunday') THEN 'Peak'
            ELSE 'Off-Peak' END AS demand
FROM market_date_info
WHERE market_day IN ('Friday', 'Saturday', 'Sunday')
  AND market_snow_flag = 0;
```

Returns 6 dates: 01-06, 01-07, 01-12, 01-13, 01-14 and 01-19. The alias can't be used in WHERE, so the condition is repeated.

Q231. In one SELECT on `market_date_info`, create three 1/0 flag columns: `is_weekend`, `is_snow_day` and `is_below_freezing`.

Answer

```
SELECT market_date,
       CASE WHEN market_day IN ('Saturday', 'Sunday') THEN 1 ELSE 0 END AS is_weekend,
       CASE WHEN market_snow_flag = 1 THEN 1 ELSE 0 END                 AS is_snow_day,
       CASE WHEN market_temp_min < 32 THEN 1 ELSE 0 END                 AS is_below_freezing
FROM market_date_info;
```

In MySQL the bare comparison alone would also work, for example `market_temp_min < 32 AS is_below_freezing`.

Q232. Sort booths so that all Large booths (`'10x10'`) come first in **descending** price order, followed by all other booths in ascending booth number. Use CASE in ORDER BY.

Answer

```
SELECT * FROM booth
ORDER BY CASE WHEN booth_size = '10x10' THEN 0 ELSE 1 END,
         CASE WHEN booth_size = '10x10' THEN booth_price END DESC,
         booth_number;
```

The first key puts the Large booths first. The second sorts only the Large booths by price (it is NULL for the others). The third orders the remaining booths by number. The result starts 19, 16, 13, 10, 7, 4, 1, then 2, 3, 5, 6 and so on.

Q233. You created a CASE column called `bill_tier` in the SELECT list. Why does `WHERE bill_tier = 'High'` fail? Write a working query that returns only the 'High' bills without using a subquery.

Answer

WHERE runs **before** SELECT, so the alias `bill_tier` doesn't exist yet. Repeat the expression in WHERE:

```
SELECT customer_id, quantity * unit_customer_price AS total_amount,
       CASE WHEN quantity * unit_customer_price > 500  THEN 'High'
            WHEN quantity * unit_customer_price >= 200 THEN 'Medium'
            ELSE 'Low' END AS bill_tier
FROM customer_purchases
WHERE quantity * unit_customer_price > 500;
```

Returns 5 rows. As a MySQL-specific extension, `HAVING bill_tier = 'High'` also works without GROUP BY, but it isn't portable.

Q234. Flag each customer as `'Check Zip'` if the zip code starts with `'0'` or is not exactly 5 characters long, otherwise `'Valid'`.

Answer

```
SELECT customer_first_name, customer_zip,
       CASE WHEN LEFT(customer_zip, 1) = '0' OR LENGTH(customer_zip) <> 5
            THEN 'Check Zip' ELSE 'Valid' END AS zip_status
FROM customer;
```

Jack and Tom are flagged.

Q235. Label each market date `'First Half'` (day 1–15) or `'Second Half'` (day 16 onwards), using RIGHT on the date.

Answer

```
SELECT market_date,
       CASE WHEN RIGHT(market_date, 2) + 0 <= 15 THEN 'First Half'
            ELSE 'Second Half' END AS month_half
FROM market_date_info;
```

Returns 15 dates in the first half and 5 in the second half.

Q236. Add a 5% night surcharge to purchases made before 06:00 or after 21:00. Show `total_amount`, a `surcharge` column (0 when none applies) and `final_amount`, rounded to 2 decimals.

Answer

```
SELECT customer_id, transaction_time,
       quantity * unit_customer_price AS total_amount,
       ROUND(CASE WHEN transaction_time < '06:00:00' OR transaction_time > '21:00:00'
                  THEN quantity * unit_customer_price * 0.05 ELSE 0 END, 2) AS surcharge,
       ROUND(quantity * unit_customer_price *
             CASE WHEN transaction_time < '06:00:00' OR transaction_time > '21:00:00'
                  THEN 1.05 ELSE 1 END, 2) AS final_amount
FROM customer_purchases;
```

8 rows get the surcharge: 22:00, 23:00 and 00:00–05:00. The purchase at exactly 21:00 does not.

Section 13: Scenario-Based Questions

Q237. Data check on `customer_purchases`: find the rows where `product_id`, `vendor_id` and `customer_id` are **not** all equal to each other.

Answer

```
SELECT * FROM customer_purchases
WHERE NOT (product_id = vendor_id AND vendor_id = customer_id);
```

Returns no rows: in every row the three ids are equal. That pattern is a sign of generated or dummy data, which is worth pointing out in an interview.

Q238. Create a username for every customer: the first 3 letters of the first name, the last 3 letters of the last name and the last 2 digits of the zip, all in lowercase (e.g. `alison45`).

Answer

```
SELECT LOWER(CONCAT(LEFT(customer_first_name, 3),
                    RIGHT(customer_last_name, 3),
                    RIGHT(customer_zip, 2))) AS username
FROM customer;
```

Examples: `alison45` (Alice Johnson, 12345) and `bobith56`.

Q239. For Type A vendors with `vendor_id` between 5 and 15, produce the sentence **"Vendor 7 (Type A) is owned by E. Jones"**.

Answer

```
SELECT CONCAT(vendor_name, ' (', vendor_type, ') is owned by ',
              LEFT(vendor_owner_first_name, 1), '. ', vendor_owner_last_name) AS
vendor_sentence
FROM vendor
WHERE vendor_type = 'Type A' AND vendor_id BETWEEN 5 AND 15;
```

Returns vendors 5, 7, 9, 11, 13 and 15.

Q240. From `market_date_info`, create a week label in the format **"W01-2024"**, zero-padding the week number to 2 digits. Show only distinct labels.

Answer

```
SELECT DISTINCT CONCAT('W', RIGHT(CONCAT('0', market_week), 2), '-', market_year) AS week_label
FROM market_date_info;
```

Returns W01-2024, W02-2024 and W03-2024.

Q241. From `vendor_inventory`, find the rows where `vendor_id` is odd and the original price is between 20 and 30. Add a CASE column `value_band` based on inventory value: `'Small'` < 1000, `'Medium'` 1000–3000, `'Large'` > 3000.

Answer

```
SELECT vendor_id, quantity, original_price,
       quantity * original_price AS inventory_value,
       CASE WHEN quantity * original_price < 1000  THEN 'Small'
            WHEN quantity * original_price <= 3000 THEN 'Medium'
            ELSE 'Large' END AS value_band
FROM vendor_inventory
WHERE vendor_id % 2 = 1
  AND original_price BETWEEN 20 AND 30;
```

Returns vendors 7, 9 and 11 as Medium, and 13 and 15 as Large.

Q242. For all booths except Medium booths, split `booth_size` into width and length, calculate the area and the price per sq ft (2 decimals), and label the booth `'Good Value'` if the price per sq ft is below 2, otherwise `'Pricey'`.

Answer

```
SELECT booth_number, booth_size, booth_price,
       LEFT(booth_size, INSTR(booth_size, 'x') - 1)                    AS width,
       SUBSTRING(booth_size, INSTR(booth_size, 'x') + 1)               AS length,
       LEFT(booth_size, INSTR(booth_size, 'x') - 1)
         * SUBSTRING(booth_size, INSTR(booth_size, 'x') + 1)           AS area,
       ROUND(booth_price / (LEFT(booth_size, INSTR(booth_size, 'x') - 1)
         * SUBSTRING(booth_size, INSTR(booth_size, 'x') + 1)), 2)      AS price_per_sq_ft,
       CASE WHEN booth_price / (LEFT(booth_size, INSTR(booth_size, 'x') - 1)
                 * SUBSTRING(booth_size, INSTR(booth_size, 'x') + 1)) < 2
            THEN 'Good Value' ELSE 'Pricey' END                         AS value_label
FROM booth
WHERE booth_description <> 'Medium booth';
```

Returns 13 booths. All the Large booths (1.00–1.90 per sq ft) are Good Value, and all the Small booths (3.06–5.14) are Pricey.

Q243. Show each customer's name masked as **"A J**"**: the first letter of each name followed by stars for the remaining characters.

Answer

```
SELECT CONCAT(LEFT(customer_first_name, 1), REPEAT('*', LENGTH(customer_first_name) - 1), ' ',
              LEFT(customer_last_name, 1),  REPEAT('*', LENGTH(customer_last_name) - 1)) AS
masked_name
FROM customer;
```

Q244. Create a receipt line for each purchase in this format: `Receipt #0001 | 2024-01-01 10:00 | Qty 2 x $20.00 = $40.00` The receipt number is `customer_id` zero-padded to 4 digits, and the time shows hours and minutes only.

Answer

```
SELECT CONCAT('Receipt #', RIGHT(CONCAT('0000', customer_id), 4),
              ' | ', market_date, ' ', LEFT(transaction_time, 5),
              ' | Qty ', quantity, ' x $', unit_customer_price,
              ' = $', quantity * unit_customer_price) AS receipt_line
FROM customer_purchases;
```

Returns `Receipt #0001 | 2024-01-01 10:00 | Qty 2 x $20.00 = $40.00`.

Q245. Find the customers whose full name, with no space, has an even number of characters **and** whose zip starts with a digit from 1 to 5.

Answer

```
SELECT customer_first_name, customer_last_name, customer_zip
FROM customer
WHERE LENGTH(CONCAT(customer_first_name, customer_last_name)) % 2 = 0
  AND LEFT(customer_zip, 1) BETWEEN '1' AND '5';
```

Returns Alice, Bob, Charlie, Eve, Leo and Nick.

Q246. Using the number extracted from `product_name`, find the products whose number is odd and between 5 and 15, and whose size is not 'Small'.

Answer

```
SELECT product_id, product_name, product_size
FROM product
WHERE SUBSTRING(product_name, INSTR(product_name, ' ') + 1) + 0 BETWEEN 5 AND 15
  AND (SUBSTRING(product_name, INSTR(product_name, ' ') + 1) + 0) % 2 = 1
  AND product_size <> 'Small';
```

Returns Products 5, 9, 11 and 15.

Q247. Find the weekday market dates where the minimum temperature in Celsius, rounded to 0 decimals, is below 0. Show the date, the day, the rounded Celsius value and a label: `'Freezing Weekday'`.

Answer

```
SELECT market_date, market_day,
       ROUND((market_temp_min - 32) * 5 / 9, 0) AS temp_min_c,
       'Freezing Weekday' AS label
FROM market_date_info
WHERE market_day NOT IN ('Saturday', 'Sunday')
  AND ROUND((market_temp_min - 32) * 5 / 9, 0) < 0;
```

Returns 10 dates (weekdays with a minimum of 31 °F or lower). A minimum of 32 °F rounds to 0, which is not below 0.

Q248. Build an `owner_display` column in the format **"DAVIS, F."** (last name in uppercase, comma, first initial). Include only vendors whose owner's last name contains a double letter or ends with `'s'`.

Answer

```
SELECT vendor_id,
       CONCAT(UPPER(vendor_owner_last_name), ', ', LEFT(vendor_owner_first_name, 1), '.') AS
owner_display
FROM vendor
WHERE vendor_owner_last_name LIKE '%ll%' OR vendor_owner_last_name LIKE '%oo%'
   OR vendor_owner_last_name LIKE '%rr%' OR vendor_owner_last_name LIKE '%ee%'
   OR vendor_owner_last_name LIKE '%ss%' OR vendor_owner_last_name LIKE '%tt%'
   OR vendor_owner_last_name LIKE '%nn%'
   OR vendor_owner_last_name LIKE '%s';
```

Returns 8 vendors: WILLIAMS, D.; JONES, E.; DAVIS, F.; MILLER, G.; MOORE, I.; THOMAS, L.; HARRIS, O.; and LEE, Q.

Q249. Find the purchases made in the second market week (7 Jan to 13 Jan 2024) between 3 PM and 9 PM. Show the day of the month and the hour (both extracted with string functions) and the total rounded to the nearest dollar.

Answer

```
SELECT customer_id,
       RIGHT(market_date, 2)          AS day_of_month,
       LEFT(transaction_time, 2)      AS hour_of_day,
       ROUND(quantity * unit_customer_price) AS total_rounded
FROM customer_purchases
WHERE market_date BETWEEN '2024-01-07' AND '2024-01-13'
  AND transaction_time BETWEEN '15:00:00' AND '21:00:00';
```

Returns 6 rows (customers 7–12, from 16:00 to 21:00).

Q250. Write the logical execution order of a query that uses FROM, WHERE, SELECT, DISTINCT and ORDER BY. Use it to explain why a SELECT alias works in ORDER BY but not in WHERE, with an example on `vendor_inventory`.

Answer

The logical order is **FROM → WHERE → SELECT (aliases created here) → DISTINCT → ORDER BY → LIMIT**.

- WHERE is evaluated before SELECT, so the alias doesn't exist yet.
- ORDER BY runs after SELECT, so the alias is available.

```
-- Fails: Unknown column 'inventory_value' in 'where clause'
SELECT vendor_id, quantity * original_price AS inventory_value
FROM vendor_inventory
WHERE inventory_value > 3000;

-- Works
SELECT vendor_id, quantity * original_price AS inventory_value
FROM vendor_inventory
WHERE quantity * original_price > 3000
ORDER BY inventory_value DESC;
```